

{System}Verilog

for ASIC/FPGA Design & Simulation

Reference:

Stuart Sutherland, E 2017,
RTL Modeling with SystemVerilog for Simulation and Synthesis: Using SystemVerilog for ASIC and FPGA Design

Abarajithan, 2025
abarakjithan07@gmail.com

About Me

Abarajithan, 2025
abarajithan07@gmail.com

- PhD Student, Computer Engineering, University of California, San Diego
- Intern @ Qualcomm (2023), AMD (2025)

Former experience:

- Undergrad Internship at CSIRO, Australia – Robotics & AI (2018)
- B.Sc. - Department of Electronic & Telecom. Eng,
University of Moratuwa, Sri Lanka (2020)
- Former Junior Lecturer (full-time), Consultant (part-time) at ENTC, UoM
- Former RTL Design Engineer at Lemurian Labs, Canada
 - Developed the compute core to accelerate AI inference & training
- abapages.com
- linkedin.com/in/abarajithan11/



- Two (slightly different) repositories
- Digital Design: github.com/abarajithan11/digital-design
 - Docker setup: works on Windows, Ubuntu, Mac
 - Has slightly simplified RTL & testbenches (may not match with slides)
 - `make sim DESIGN=alu` - simulate with verilator
 - `make gtkwave DESIGN=alu` - view waveform after simulation
 - `make gds DESIGN=alu` - ASIC flow using open-source tools
 - `make show_layout DESIGN=alu` - show chip layout
 - `DESIGN=alu` will find a file named `material/designs/*/alu.f`
 - That file has locations for all RTL and testbenches.
- SystemVerilog: github.com/SkillSurf/systemverilog
 - Contains the code from slides, but doesn't have runtime environment
 - You can copy the code from here, put into digital-design, create your own .f file, and run things.
- Message me if you have any trouble

EDA Playground

Abarajithan, 2025
abarajithan07@gmail.com

- Can quickly test SV features
- Save & share code
- Login with Google
- Test simulator support
- Industry standard simulators
- Each cost millions of dollars, but given for free
- Testbenches can be SV, C++, Python

3. Select a simulator

Cadence Xcelium shows multidimensional signals better

5. Tick this to view waveform

The screenshot displays the EDA Playground web interface. The top navigation bar includes 'Run', 'Save', and 'Copy' buttons. The left sidebar contains settings for 'Languages & Libraries', 'Testbench + Design' (set to SystemVerilog/Verilog), 'UVM / OVM' (set to None), 'Other Libraries' (including OVL 2.8.1 and SVUnit 2.11), 'Tools & Simulators' (set to Cadence Xcelium 20.09), 'Compile Options' (-timescale 1ns/1ns -sysv), 'Run Options' (-access +rw), and a checkbox for 'Open EPWave after run' which is checked. The main workspace is divided into two panels: 'testbench.sv' and 'design.sv'. The 'testbench.sv' panel contains a Verilog testbench for a full adder, with a red box highlighting the 'dumpfile' and '\$dumpvars' statements. The 'design.sv' panel contains the Verilog RTL code for a full adder. The bottom panel shows the simulation output, including the command 'xcelium> run', the completion message 'Simulation complete via \$finish(1) at time 720 NS + 0', and the command 'xcelium> exit'. The output also shows the tool version '20.09-s003' and the exit time 'Aug 31, 2022 at 08:36:56 EDT'. A red box highlights the 'Opening EPWave...' message, indicating that the waveform viewer has been launched.

6. Run

2. Write testbench

1. Write RTL

4. Dump waveforms

Log Share

xcelium> run

OK

Simulation complete via \$finish(1) at time 720 NS + 0

./testbench.sv:24 \$finish(); // simulation ends

xcelium> exit

TOOL: xrun 20.09-s003: Exiting on Aug 31, 2022 at 08:36:56 EDT (total: 00:00:01)

Finding VCD file...

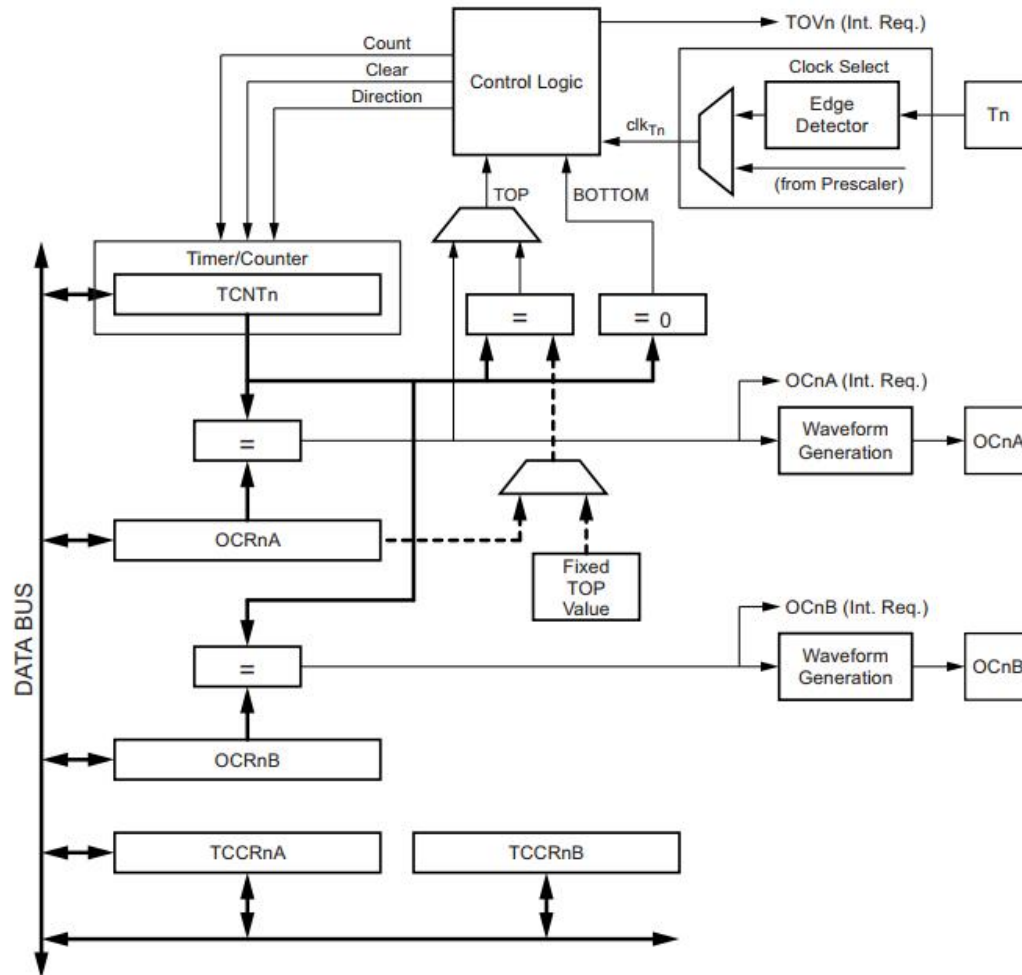
./dump.vcd

[2022-08-31 08:36:56 EDT] Opening EPWave...

Done

Why Digital Design?

Figure 17-1. 8-bit Timer/Counter Block Diagram



- Processors, GPUs, ASICs are digital circuits
- Fabricated in silicon, doped to form transistors → gates → multiplexors, registers...etc.
- RTL Design: (Register Transfer Level)**
A digital circuit is accurately described as text, using a hardware description language (HDL)
- Verification: Testing in Simulation**
Need a language to writes tests to check the behavior of designed circuits.

How do I implement my digital circuit?

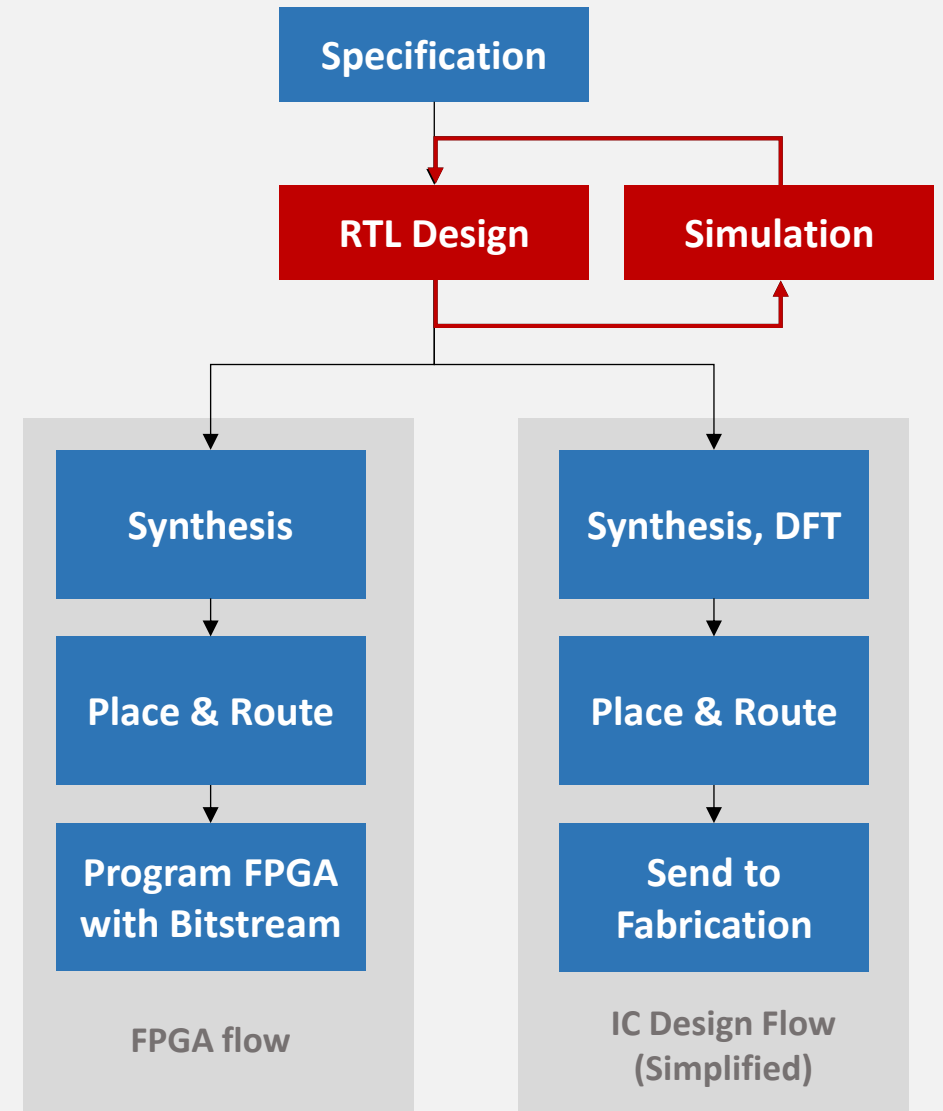
Abarajithan, 2025
abarajithan07@gmail.com

FPGA

- Generic, reconfigurable chip, field-programmable
- Free/cheaper tools (software)
- Dynamically realizes any digital circuit using look up tables and switches.
- Can change the designs quickly and iterate
- Silicon includes generic, routing logic also
- Cheaper in low volume:
Network routers, video encode/decode, satellites, scientific instruments, signal processing

ASIC

- You build your own chip out of bare, blank silicon
- If you need high volume, cheaper per-chip
- Better performance: silicon only includes logic needed for that circuit



An HDL is NOT a programming language

Abarajithan, 2025
abarajithan07@gmail.com

Programming Language (control flow)

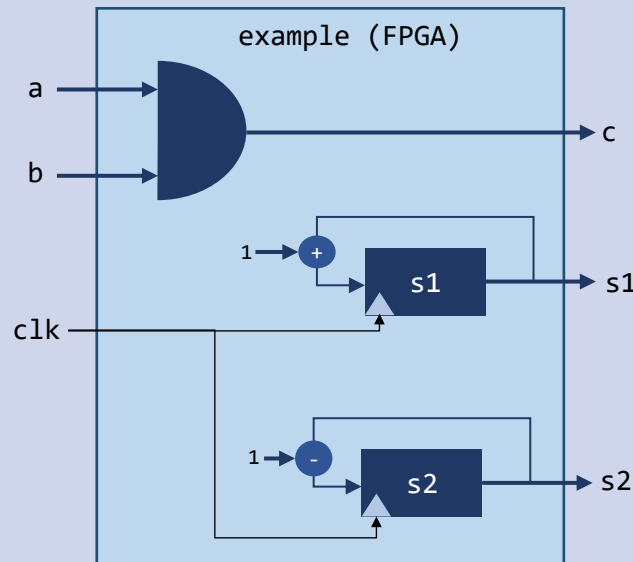
```
● a = True
  b = False
  c = a and b
  print(c)          >> False

  a = True
  b = True
  print(c)          >> False

  s1 = 0
  for i in range(3):
    s1 = s1 + 1
    print(s1)        >> 1
                    >> 2
                    >> 3

  s2 = 7
  for i in range(2):
    s2 = s2 - 1
    print(s2)        >> 6
                    >> 5
```

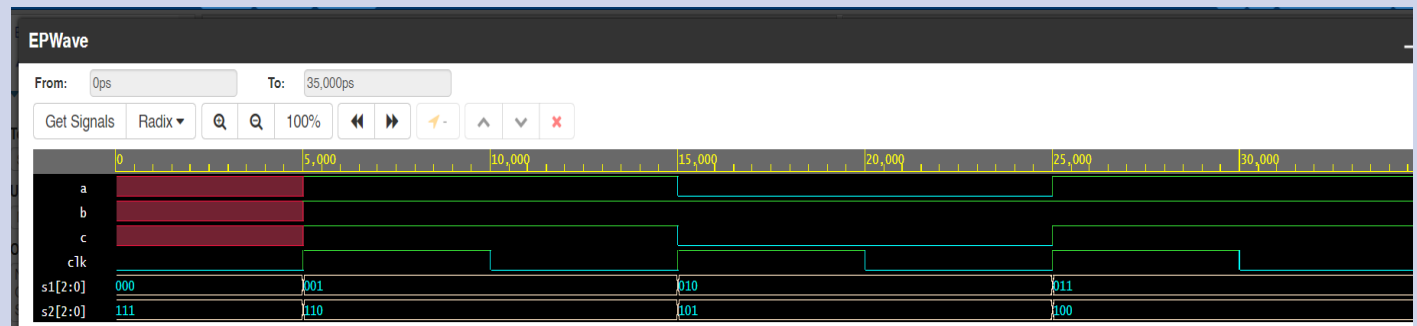
Hardware Description Language (HDL) (data flow)



```
module example (
    input logic clk,a,b,
    output logic c,
    output logic [3-1:0] s1=0,
    output logic [3-1:0] s2=7
);

    assign c = a & b;

    always_ff @(posedge clk)
        s1 <= s1 + 1;
    always_ff @(posedge clk)
        s2 <= s2 - 1;
endmodule
```



An HDL is NOT a programming language

Abarajithan, 2025
abarajithan07@gmail.com

Programming Language	Hardware Description Language (HDL)
Step by step instructs an existing processor what to do	Textual description of your circuit. We will describe our own processor with an HDL, and make it execute a programming language we created.
Control flow: Read and executed line by line	Tells the synthesis tool how to make the circuit. Data Flow: After implementation, you can give signals and get outputs
Compiling: High level code gets translated to the instructions for a particular processor family	Compiling (Synthesis): Translating a high-level design (RTL) into a low-level, target-specific design (netlist)

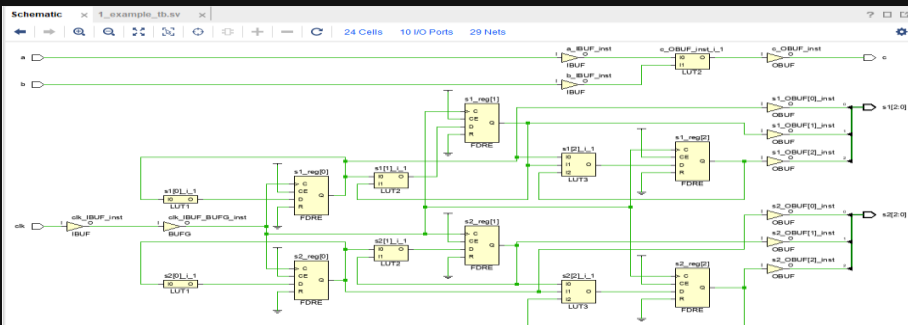
(System)Verilog is Two in One!

Abarajithan, 2025
abarajithan07@gmail.com

Synthesizable SV is an HDL

```
module example (  
    input  logic clk,a,b,  
    output logic c,  
    output logic [3-1:0] s1=0,  
    output logic [3-1:0] s2=7  
);  
  
assign c = a & b;  
  
always_ff @(posedge clk)  
    s1 <= s1 + 1;  
always_ff @(posedge clk)  
    s2 <= s2 - 1;  
always_ff@(posedge clk);  
endmodule
```

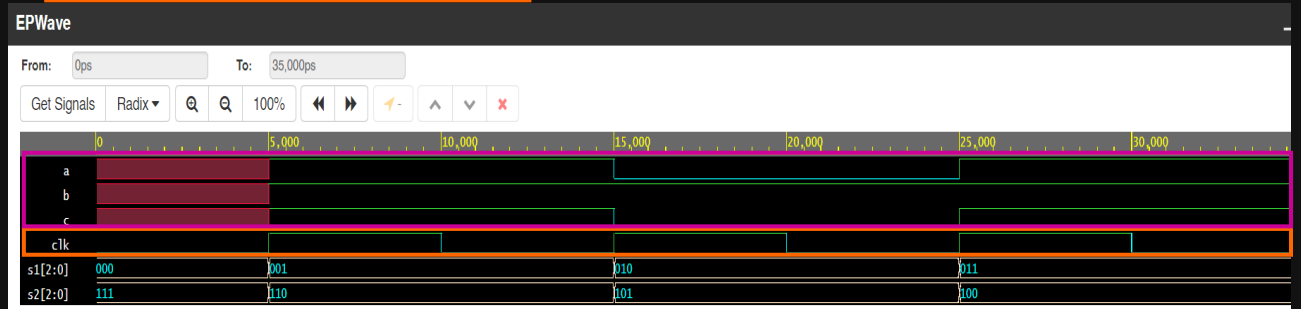
IS SYNTHESIZABLE!



SV for Verification is a Programming Language

```
module example_tb;  
    timeunit      1ns;  
    timeprecision 1ps;  
  
    logic clk, a,b,c;    forever,  
    logic [3-1:0] s1,s2;  #DELAY,  
    example dut (.*)@ (posedge clk);  
                           NOT SYNTHESIZABLE!  
  
    localparam PERIOD = 10;  
    initial begin  
        clk = 0;  
        forever  
            #(PERIOD/2)  
            clk <= ~clk;  
    end  
endmodule
```

initial begin
 \$dumpfile("dump.vcd");
 \$dumpvars(0, dut);
 @(posedge clk);
 #1 a <= 1; b <= 1;
 @(posedge clk);
 #1 a <= 0; b <= 1;
 @(posedge clk);
 #1 a <= 1; b <= 1;
 @(posedge clk);
 \$finish();
end



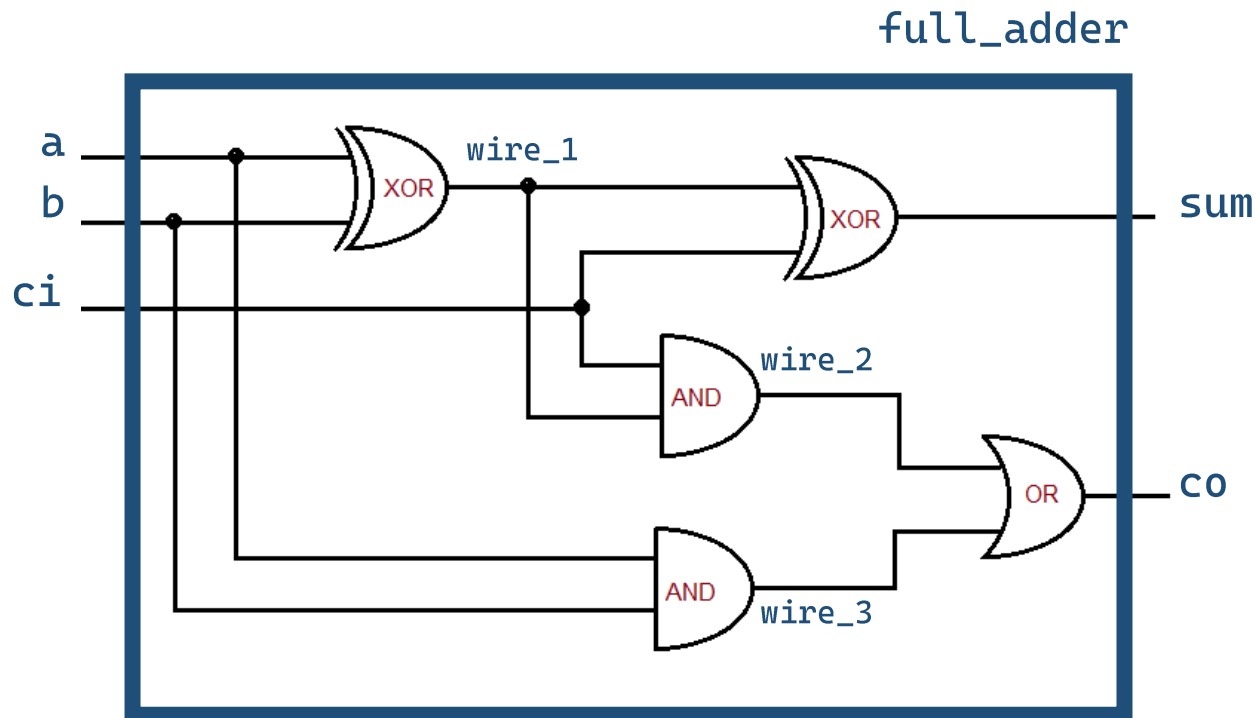
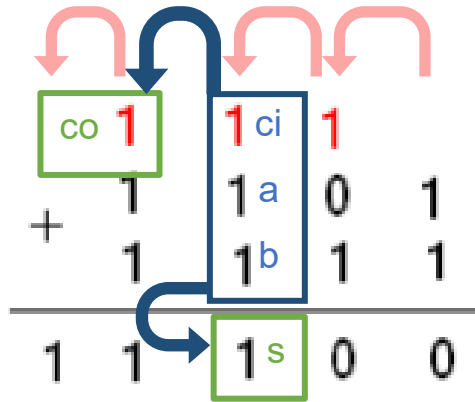
One Language, Two Objectives

Abarajithan, 2025
abarajithan07@gmail.com

Design	Verification (Simulation)
Describes a circuit • RTL – high level • Netlist – low level	• Feeds inputs to the designed circuit step by step • Gets outputs from the circuit • Compares with expected outputs to “verify”
Synthesizable into a digital circuit.	Not synthesizable , testbenches are only simulated. Runs as a software.
Read by the synthesis compiler • FPGA: Vivado/Quartus synthesis, • ASIC: Synopsys Design Compiler, Cadence Genus	Read by the simulator • FPGA: Qsim (Altera Quartus), ModelSim, Vivado • ASIC: Synopsys VCS, Cadence Xcelium

1. RTL Design: 1-bit Full Adder

Abarajithan, 2025
abarajithan07@gmail.com



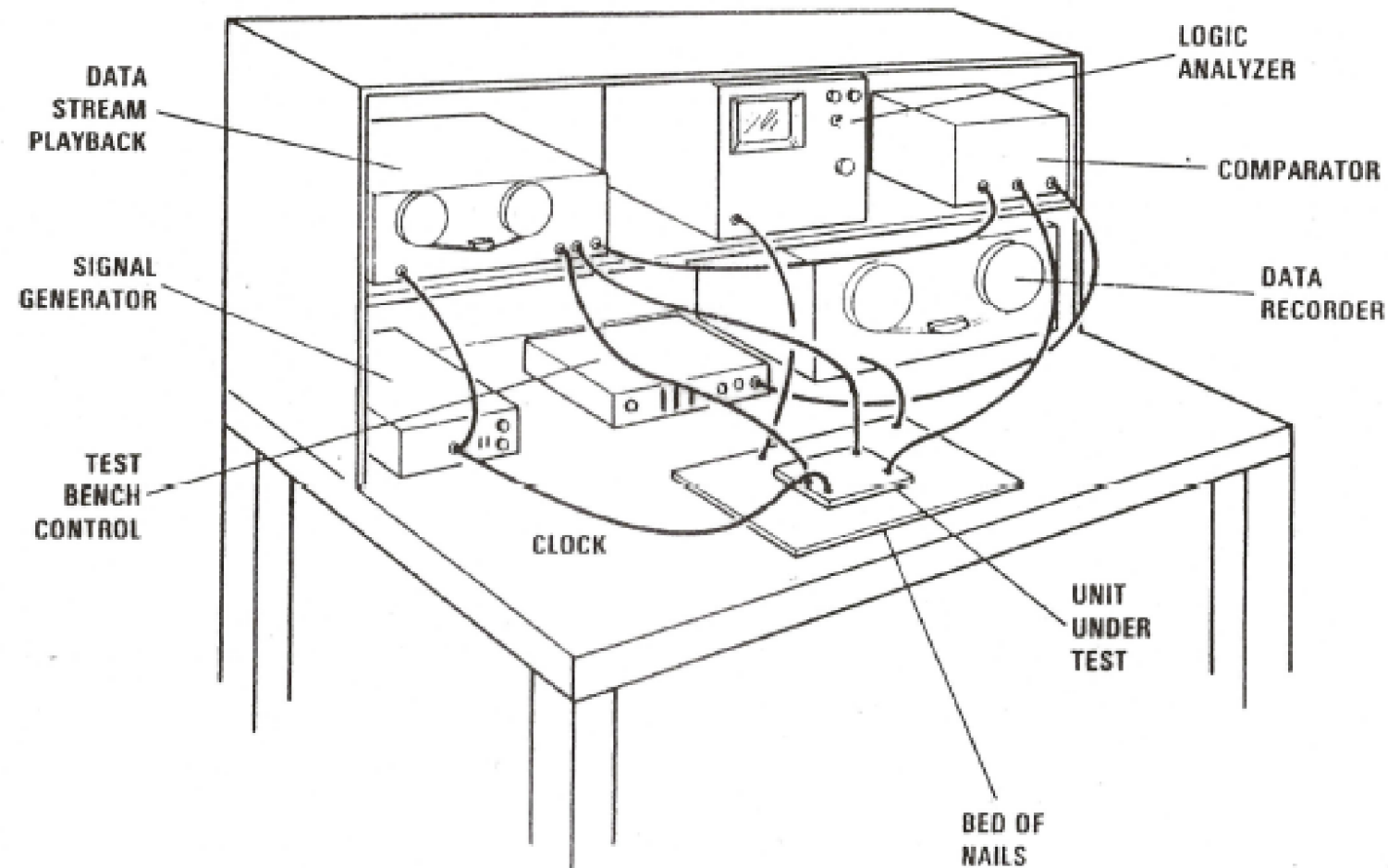
```
/*  
    This code describes a full_adder circuit  
*/  
module full_adder Module name  
(  
    input logic a, b, ci,  
    output logic sum, co  
);  
  
    logic wire_1, wire_2;  
    assign wire_1 = a ^ b; // bitwise XOR  
    assign wire_2 = wire_1 & ci; // bitwise AND  
  
    wire wire_3 = a & b; // logic wire_1 = a^b is not same  
    // bitwise AND  
  
    always_comb begin  
        co = wire_2 | wire_3; // bitwise OR  
        sum = wire_1 ^ ci; // bitwise XOR  
    end  
  
endmodule
```

Three equivalent ways of combinational assignments

What's a Testbench?

Abarajithan, 2025
abarajithan07@gmail.com

The Test Bench Concept



What's a Testbench?



- Software code, written using **non-synthesizable** features of SystemVerilog in order to test (verify) the behavior our design in simulation

`#delay, randomize(), $fdisplay(), $fopen()`

- Each `initial` block **starts independently** (in parallel) and **runs sequentially**
- Simulators: Qsim (Altera Quartus), ModelSim, Vivado Simulator
 - Assertions: automatically compare behavior against a reference model
 - Display waveforms to debug clock by clock
 - File input / output

1. Testbench & Simulation: 1-bit Full Adder

Abarajithan, 2025
abarakjithan07@gmail.com

`module full_adder_tb;` No need inputs / outputs. Software, not hardware

`timeunit 10ns; timeprecision 1ns;` Time units

`logic a=0, ci=0, b, sum, co;` Variable declaration, with and without initial values

`full_adder dut (.*);` Module instantiation: (dot-star way)

`initial begin // simulation starts`

0 `$dumpfile("dump.vcd"); $dumpvars(0, dut);` Dump waveforms

1 `#30 a <= 0; b <= 0; ci <= 0;` delay/wait and drive input pins

2 `#10 a <= 0; b <= 0; ci <= 1;`

3 `#20 a <= 1; b <= 1; ci <= 0;`

Concatenation

`#1 assert ({co,sum} == a+b+ci)` Assertion (condition)
`Concat $display ("OK");` pass statement;
`else $error("Not OK");` fail statement;

4 `#10 a <= 1; b <= 1; ci <= 1;`

Accessing deeper signals

Assertion – only fail

`#1 assert (dut.wire_1 == 0) // 1 (xor) 1 = 0`
`else $error("False. wire_1:%d", dut.wire_1);`

`$finish();` // simulation ends String formatting with integer argument

`end` Stop simulation without errors

`endmodule`

IPWave

From 0ps To: 710ps

Get Signals

Radix

100%

100%

100%

100%

100%

100%

100%

100%

100%

100%

100%

100%

100%

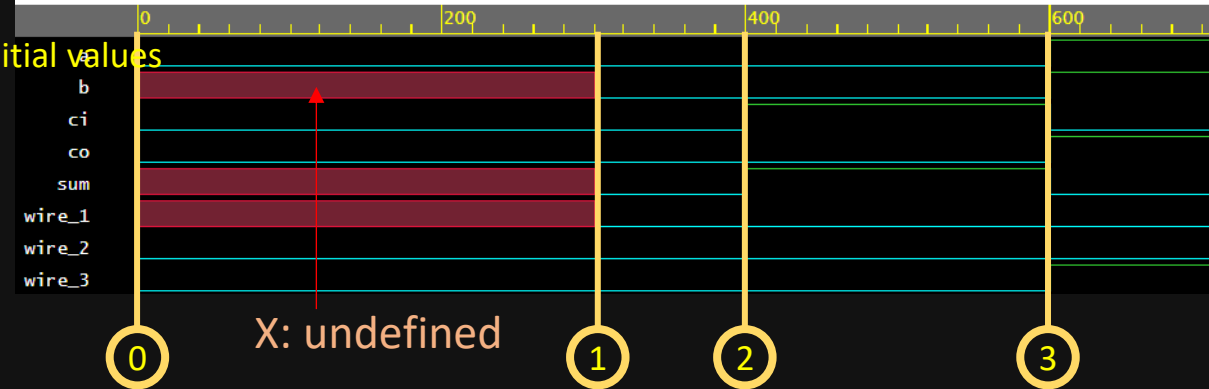
100%

100%

100%

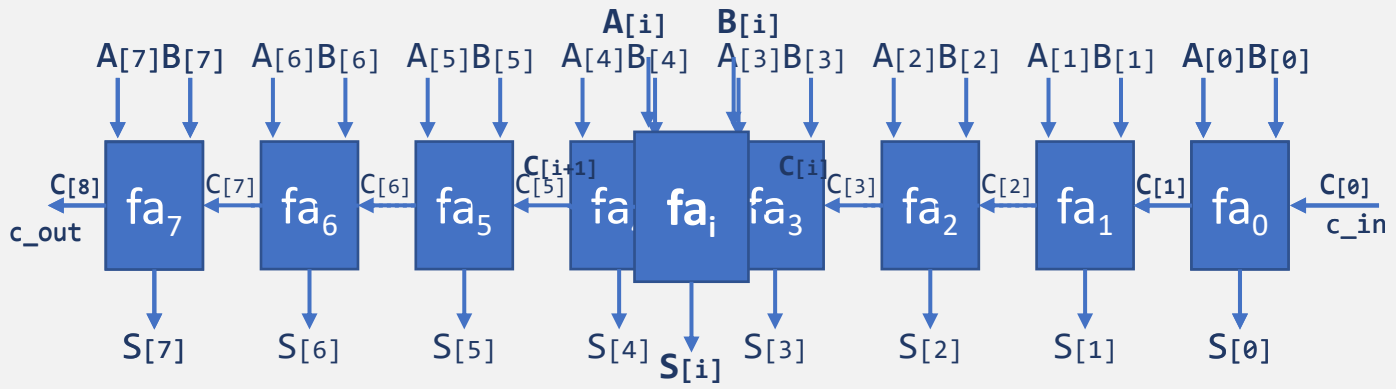
100%

100%



	A	B	Cin	Sum	Cout
1	0	0	0	0	0
2	0	0	1	1	0
	0	1	0	1	0
	0	1	1	0	1
	1	0	0	1	0
	1	0	1	0	1
3	1	1	0	0	1
4	1	1	1	1	1

2. N-bit Full Adder



LITTLE ENDIAN	BIG ENDIAN
- Use this to represent numbers - Read left to right (□) - LSB is rightmost $0000_1110_2 = 14_{10}$ - N-bit vector (wire) declared as: <code>logic [N-1 : 0] wire_1;</code>	- Read right to left (□) - LSB is leftmost $0000_1110_2 = 112_{10}$ - N-bit vector (wire) declared as: <code>logic [0 : N-1] wire_1;</code>

```
module n_adder #(
    parameter N=8 // Parameterized module
)(
    signed, N-bit, little endian wire vectors, packed
    input logic signed [N-1:0] A, B,
    input logic ci,
    output logic signed [N-1:0] S,
    output logic co
);
    logic C [N:0]; // (N+1)-bit, little endian wire array, unpacked
    assign C[0] = ci; // Referencing certain bits
    assign co = C[N];
// Generating instances
    genvar i;
    for (i=0; i<N; i=i+1) begin:add
        // full_adder fa (A[i],B[i],C[i],C[i+1],S[i]);
        full_adder fa (
            .a (A[i ]),
            .b (B[i ]),
            .ci (C[i ]),
            .co (C[i+1]),
            .sum (S[i ])
        );
    end
endmodule
```

2. Testbench & Simulation: 8-bit Full Adder

Abarajithan, 2025
abarajithan07@gmail.com

```
module n_adder_tb;
    timeunit 1ns;
    timeprecision 1ps;

    localparam N = 8;

    logic signed [N-1:0] A, B, S;
    logic ci, co;

    n_adder #(.N(N)) dut (.*);

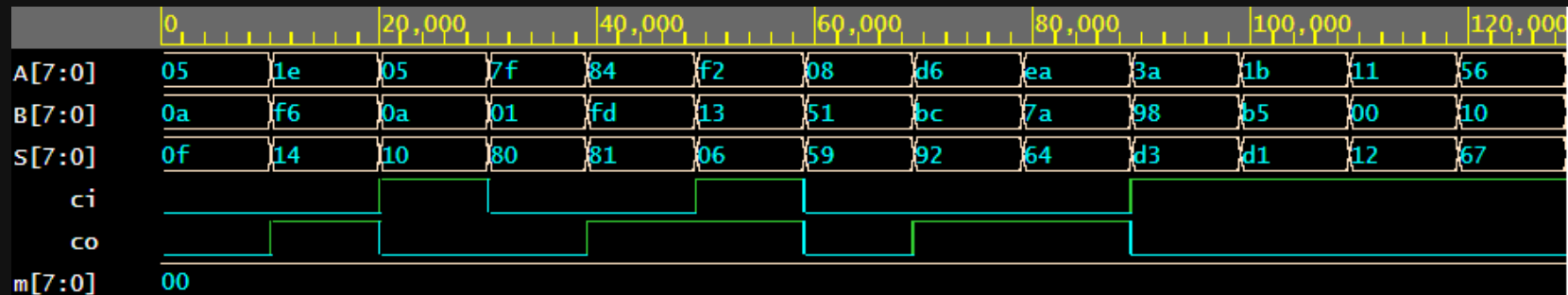
initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0, dut);
end
```

```

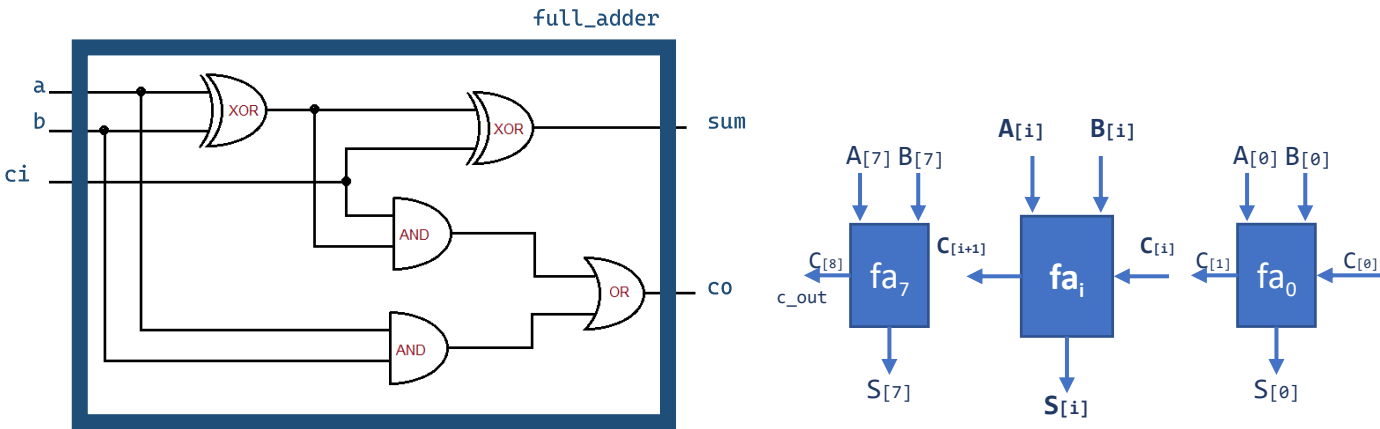
A <= 8'd5; B <= 8'd10; ci <= 0;
#1 assert (S == 8'd15) else $error("Fail");
    Delay to allow simulator to send value into DUT
#10 A <= 8'd30; B <= -8'd10; ci <= 0;
#10 A <= 8'd5; B <= 8'd10; ci <= 1;
#10 A <= 8'd127; B <= 8'd1; ci <= 0;
S; repeat loop
repeat(10) begin
    #9 Randomize with constraints
    std::randomize(ci);
    std::randomize(A) with { A inside {[-128:127]}; };
    std::randomize(B) with { B inside {[-128:127]}; };
    #1
    assert ({co,S} == A+B+ci)
        else $error("%d+%d+%d != {%d,%d}", A,B,ci,co,S);
end
end
endmodule

```

- Radix
- View internal signals
- Arrange signals



Adder – Let synthesis tool do its job



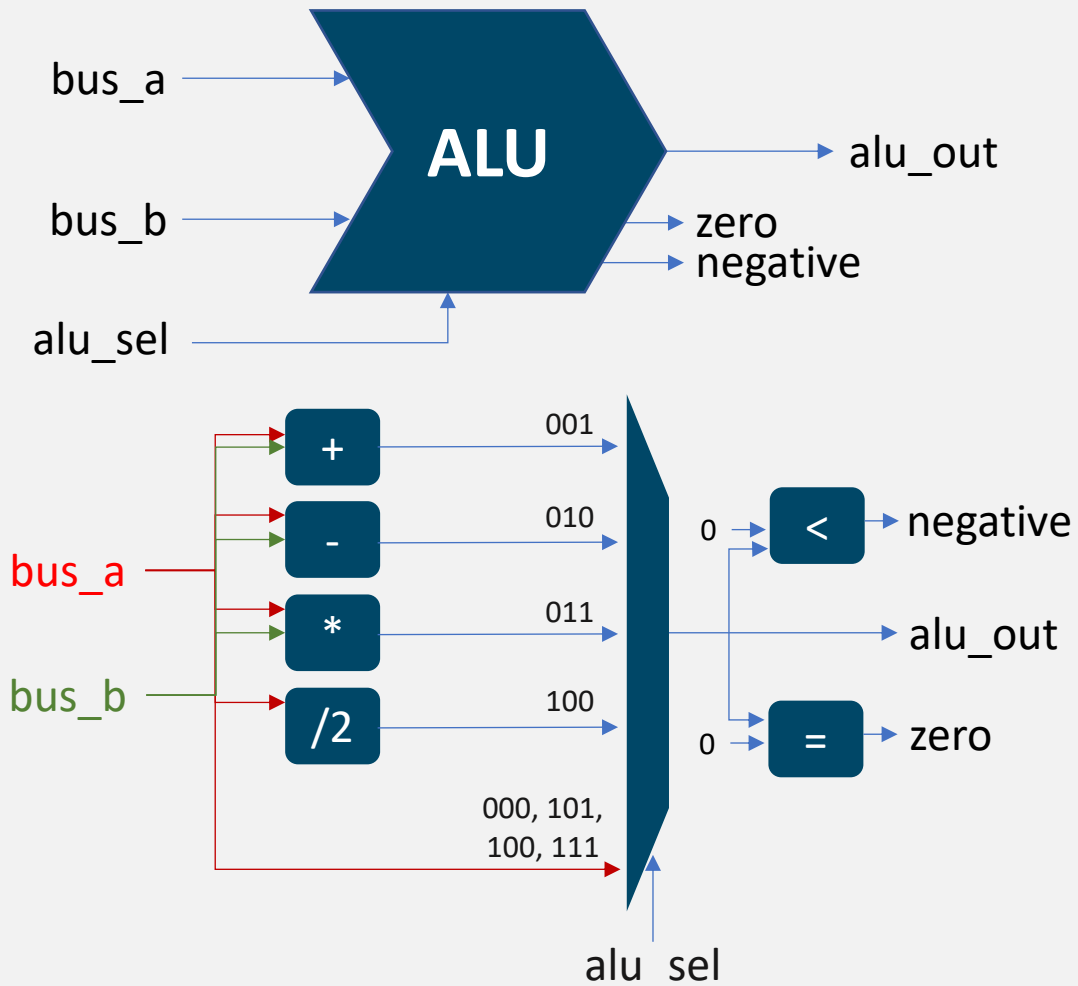
- There are several adder designs.
- Ripple carry is the worst – delay
- Just use the (+) operator – synthesis tool will choose a better implementation
- RTL = high level design
- For cutting edge applications, circuits could be manually optimized

```
module full_adder (  
    input logic a, b, ci,  
    output logic sum, co  
);  
    assign {co, sum} = a + b + ci;  
endmodule
```

```
module n_adder #(  
    parameter N=4  
)(  
    input logic signed [N-1:0] A, B,  
    input logic ci,  
    output logic signed [N-1:0] S,  
    output logic co  
);  
    assign {co, S} = A + B + ci;  
endmodule
```

3. Combinational ALU

We are building the ALU for our CPU



```
module alu #(
    parameter WIDTH      = 8,
              W_ALU_SEL  = 3
)(
    input  logic signed [WIDTH-1:0] bus_a,
    input  logic signed [WIDTH-1:0] bus_b,
    output logic signed [WIDTH-1:0] alu_out,
    input  logic [W_ALU_SEL  -1:0] alu_sel,
    output logic zero, negative
);
```

always_comb

```
unique case (alu_sel)
    'b001 : alu_out = bus_a + bus_b;
    'b010 : alu_out = bus_a - bus_b;
    'b011 : alu_out = bus_a * bus_b;
    'b100 : alu_out = bus_a/2;
    default: alu_out = bus_a; // pass a
endcase
```

case -> multiplexer
unique -> fully parallel

```
assign zero      = (alu_out == 0);
assign negative  = (alu_out < 0);
```

endmodule

3. Combinational ALU - Simulation

Class

```
class Random_Sel; Random Attribute
    rand bit [2:0] num;
    constraint c {num inside {[0:4]}};

    function new();
        this.num = 3'd4;
    endfunction
endclass
```

Constraints for Randomization
Optional Constructor

```
class Random_Num #(WIDTH=8);
    rand bit signed [WIDTH-1:0] num;
endclass 2-state logic: {0,1}, without X,Z

module alu_tb;
    timeunit 1ns/1ps;

    localparam WIDTH = 8;
    logic [2:0] sel;
    logic signed [WIDTH-1:0] bus_a, bus_b, alu_out;
    logic Z;

    alu #(.WIDTH(WIDTH)) dut (.*);
```

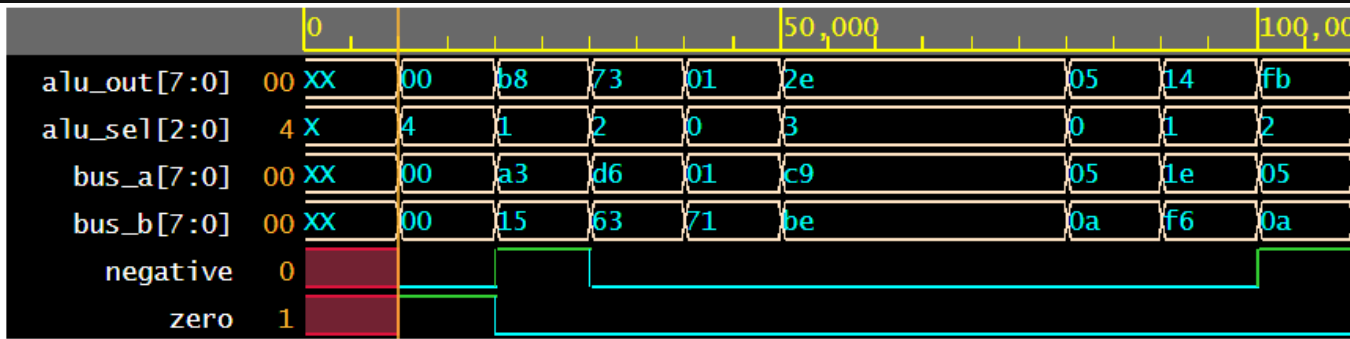
```
Random_Num #(.WIDTH(WIDTH)) A_r = new(), B_r = new();
Random_Sel sel_r = new();

initial begin
    $dumpfile("dump.vcd"); $dumpvars(0, dut);

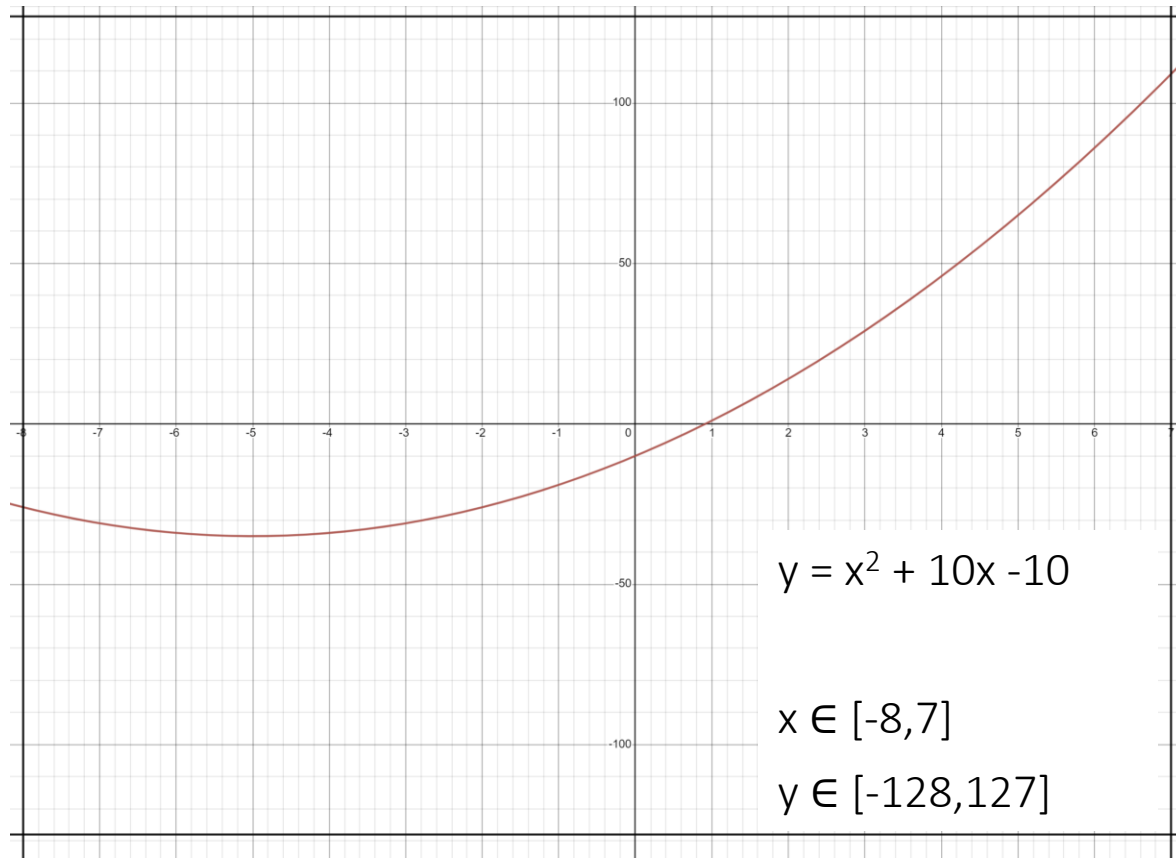
    repeat(5) begin
        #10
        bus_a <= A_r.num; bus_b <= B_r.num; sel <= sel_r.num;
        A_r.randomize();
        B_r.randomize();
        sel_r.randomize();
    end

    #30 bus_a <= 8'd5; bus_b <= 8'd10; sel <= 3'b000;
    #10 bus_a <= 8'd30; bus_b <= -8'd10; sel <= 3'b001;
    #10 bus_a <= 8'd5; bus_b <= 8'd10; sel <= 3'b010;
    #10 bus_a <= 8'd51; bus_b <= 8'd17; sel <= 3'b011;
end
endmodule
```

Object handle (pointer)
Object "constructed"
Randomized at construction, custom constructor
Accessing an attribute
Randomizing rand, randc
attributes as per constraints



4. Functions & Lookup Tables



- Factorial?
- Quadratic?
- Inverse of a complex function?

LUTS

- Read only, hardcoded values
- “memorize” a function

Functions

- Synthesizable into combinational circuits
- Or, can be used to fill LUTs

Tasks

- For simulation, can have delays (#5, @posedge clk)

4. Functions & Lookup Tables

Abarajithan, 2025
abarajithan07@gmail.com

```
module function_lut #(
    parameter W_X=4, W_Y=8,
    parameter signed A= 1, B= 10, C= -10
)(
    input  logic signed [W_X-1:0] xf, xq,
    input  logic signed [W_Y-1:0] yiq,
    output logic signed [W_Y-1:0] yf, yq_lut, yq_fun,
    output logic signed [W_X-1:0] xiq
);
```

Static vs automatic (synthesizable)

Output size

```
function automatic [W_Y-1:0] factorial(
    input logic [W_X-1:0] n);
    factorial = (n==1 || n==0) ? 1 : n * factorial(n-1);
endfunction
```

Factorial

Can be recursive

```
function automatic signed [W_Y-1:0] quadratic(
    input logic signed [W_X-1:0] x);
    return A * x**2 + B * x + C;
endfunction
```

Quadratic

```
function automatic signed [W_X-1:0] inverse_quadratic(
    input logic signed [W_Y-1:0] y);
    logic signed [W_Y-1:0] result, quad;
    int error, min_error = 2**W_Y;
    Check all possible inverse values
```

Inverse quadratic

```
    for (int ux=0; ux<2**W_X; ux++) begin
        logic signed [W_X-1:0] x = ux; // signed cast
        quad = quadratic(x);
```

```
        error = (y - quad);
        error = error > 0 ? error : -error;
```

Find the closest inverse (minimum error)

```
        if (error < min_error) begin
            min_error = error;
            result = x;
        end
```

```
    end
    return result;
endfunction
```

// LUT Generation

```
genvar i;
logic signed [2**W_X-1:0][W_Y-1:0] lut_factorial;
logic signed [2**W_X-1:0][W_Y-1:0] lut_quadratic;
logic signed [2**W_Y-1:0][W_X-1:0] lut_inverse_quadratic;
```

Input size

Output size

LUTs can be either packed or unpacked

```
for (i=0; i<2**W_X; i=i+1)
    assign lut_factorial[i] = factorial(i);
```

Fill up the LUT for all possible inputs

```
for (i=0; i<2**W_X; i=i+1)
    assign lut_quadratic[i] = quadratic(i);
```

```
for (i=0; i<2**W_Y; i=i+1)
    assign lut_inverse_quadratic[i] = inverse_quadratic(i);
```

// LUT usage

```
assign yf = lut_factorial[xf];
assign xiq = lut_inverse_quadratic[yiq];
assign yq_lut = lut_quadratic[xq];
```

Multiplexer to read LUT values

```
assign yq_fun = quadratic(xq);
```

Not a LUT. Function synthesized

endmodule

`timescale 1ns/1ps

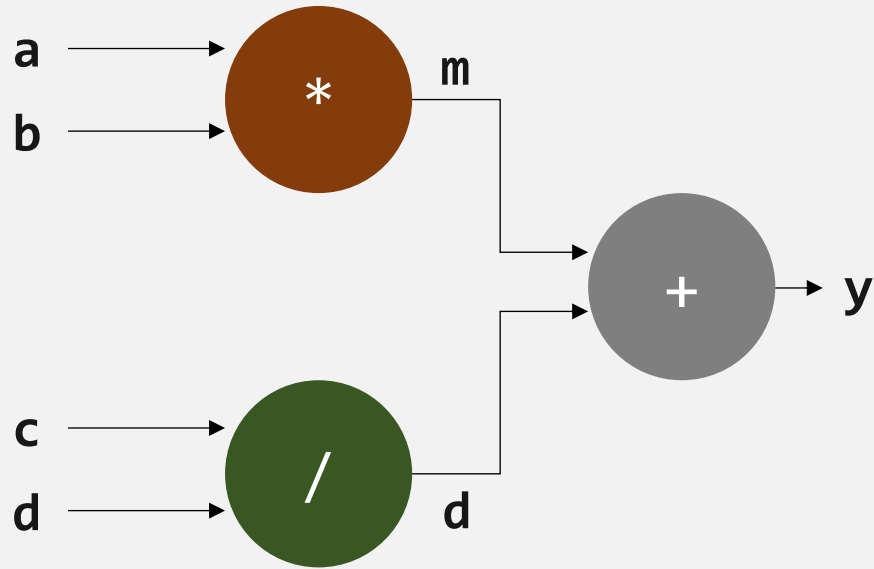
Testbench

```
module function_lut_tb;
    localparam W_X=4, W_Y=8;
    logic signed [W_X-1:0] xf = 3, xq = 3, xiq;
    logic signed [W_Y-1:0] yiq = 3, yf, yq_lut, yq_fun;
    function_lut #(.W_X(W_X), .W_Y(W_Y)) dut (.*);
```

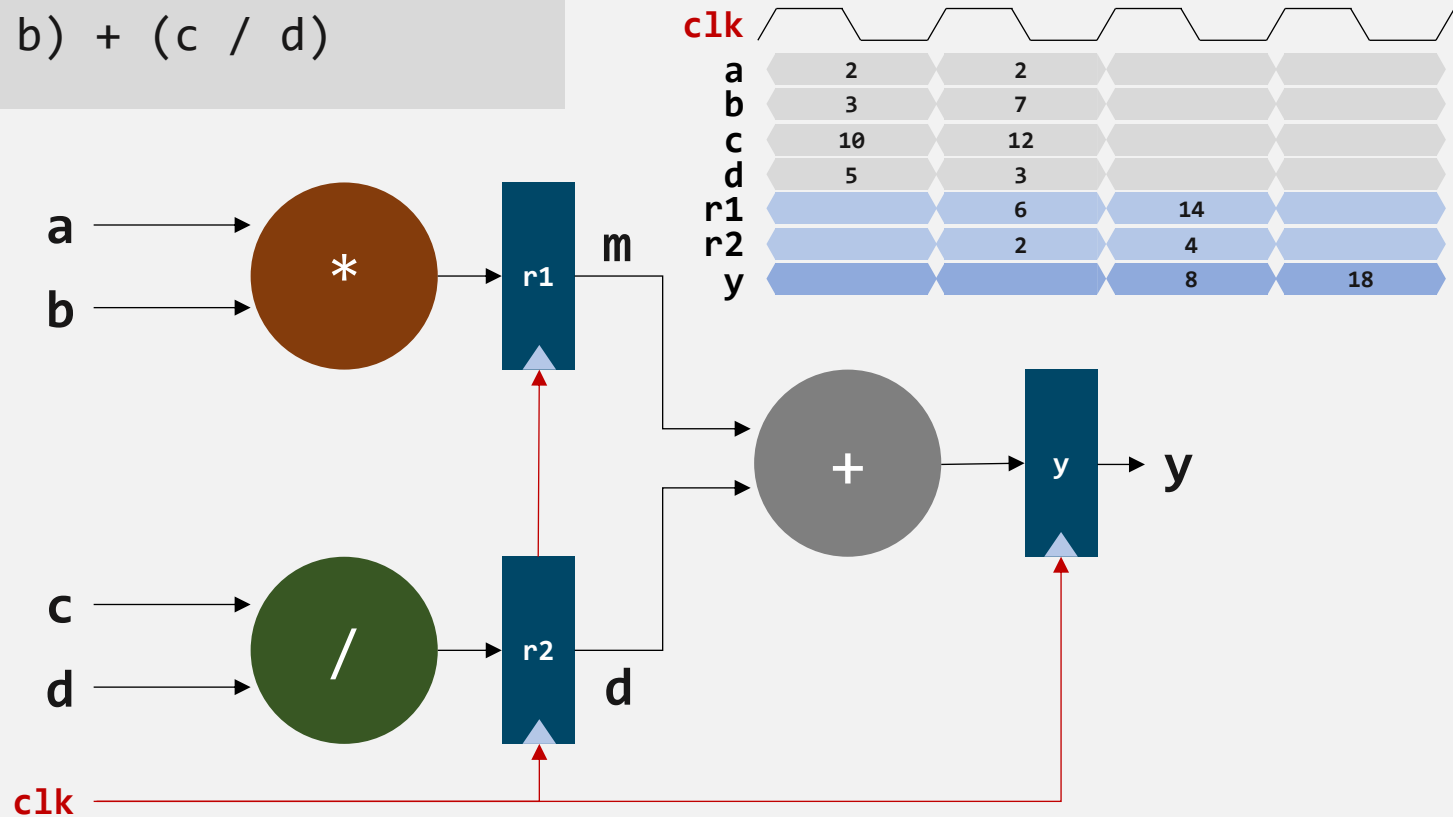
```
    initial begin
        $dumpfile("dump.vcd"); $dumpvars;
        #20 $finish;
    end
endmodule
```

Why clock?

$$y = (a * b) + (c / d)$$

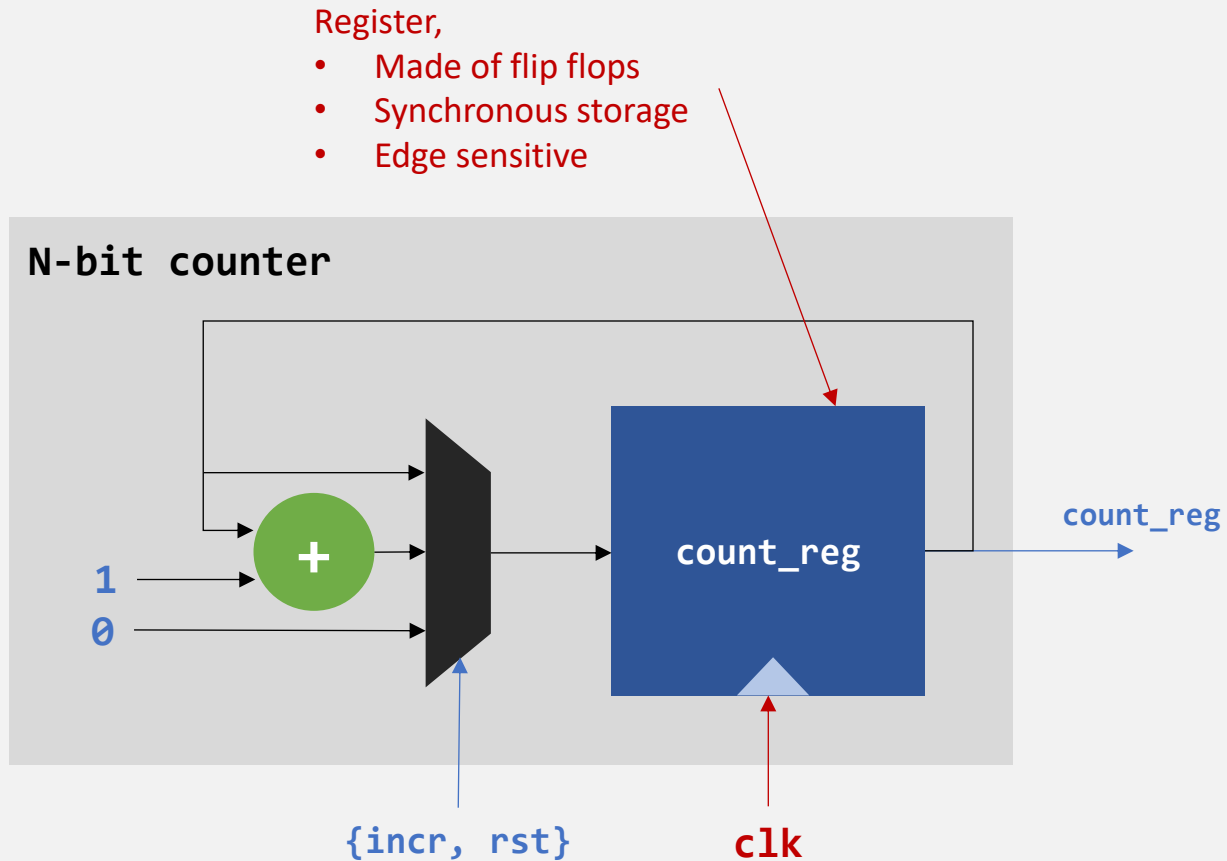


- Signals take time to run through wires
- Different delays in multiplier and divider
- m, d stabilize at different times
- We are not sure:
 - when to read y
 - when to change a,b,c,d
- Difficult to design anything complex



- Registers, made of flip flops are sequential. They capture data at an **edge of clock signal** and hold it unchanged through the cycle.
- Synthesis, P&R software will try, such that all signals take less than a clock period (which we set) to go from one register to another.
- Each signal should stabilize before next posedge, to be accurately read.

4. Sequential Circuits – Counter



```
module counter #(parameter N = 8)
(
    input logic clk, rst, incr,
    output logic [N-1:0] count_reg
);
```

Flip flop sensitive to clk's positive edge

```
always_ff @(posedge clk) begin
    if (rst ) count_reg <= 0;
    else if (incr) count_reg <= count_reg + 1'b1;
end
endmodule
```

non-blocking assignment

else can be skipped for
sequential circuits (always_ff).
No latch inferred here.

4. Counter - Simulation

```
module counter_tb;
```

```
    timeunit 10ns/1ps;
    localparam WIDTH = 8;
```

```
    logic clk=0, rst, incr;
    logic [WIDTH-1:0] count_reg;
```

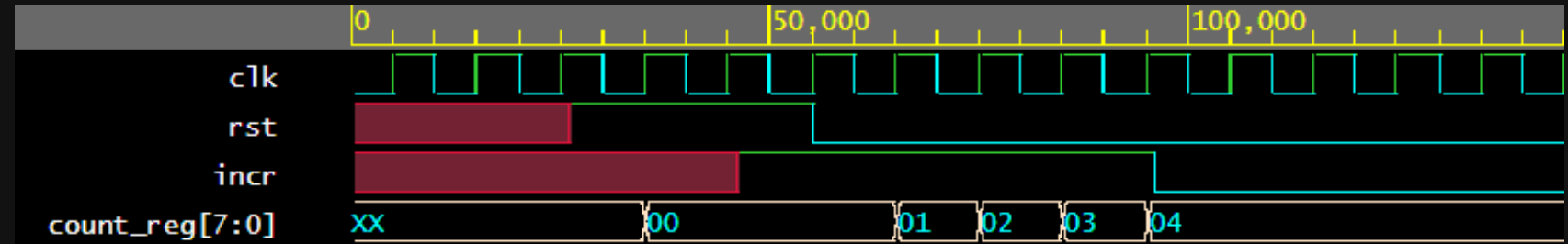
```
    counter #(.WIDTH(WIDTH)) dut (.*);
```

```
    localparam CLK_PERIOD = 10;
    initial forever
        #(CLK_PERIOD/2) clk <= ~clk;
```

Clock generation

```
    initial begin
        $dumpfile("dump.vcd");
        $dumpvars(0, dut);
```

```
        #(CLK_PERIOD * 2)
        @(posedge clk);
        #1 rst <= 1;
```



```
        repeat(2) @(posedge clk);
```

```
        #1 incr <= 1;
```

```
        @(posedge clk);
```

```
        #1 rst <= 0; incr <= 1;
```

ALWAYS assign after a delay. else weird behavior

```
        repeat(4) @(posedge clk);
```

```
        #1 incr <= 0;
```

```
        repeat(4) @(posedge clk);
```

```
        #1 rst <= 1;
```

```
        $finish();
```

end

endmodule

- Reset has priority
if() ... else ...
priority case () ...

- unique case() ...
 - Makes all parallel
 - No priority
 - Assumes the FSM won't go to any other state

- Sync resets for FPGA, Async reset for ASIC

Synchronous Circuits: Flipflops & Latches

Abarajithan, 2025
abarajithan07@gmail.com

Latch

- Transparent when en==1
- “latches” to x (remembers it) when en==0
- Level-sensitive
- Not advisable to use

Flip flop (register)

- Edge sensitive (posedge or negedge)
- Built using two latches
- Remembers state, counts...etc, used for pipeline

Reset

- Active low, asynchronous preferred for ASIC
- Active high, synchronous preferred for most FPGAs (check datasheet)

```
//latch
always_latch
    if (en) y <= x;

// flip flop / register
always_ff @(posedge clk)
    y <= x;

// register with active low, async reset
always_ff @(posedge clk or negedge rstn)
    if (!rstn) y <= '0;
    else y <= x;

// register with active high, sync reset
always_ff @(posedge clk)
    if (rst) y <= '0;
    else y <= x;

// register with clock enable
always_ff @(posedge clk or negedge rstn)
    if (!rstn) y <= '0;
    else if (en) y <= x;
```

FIR Filter – What & Why

Abarajithan, 2025
abarajithan07@gmail.com

Digital Signals

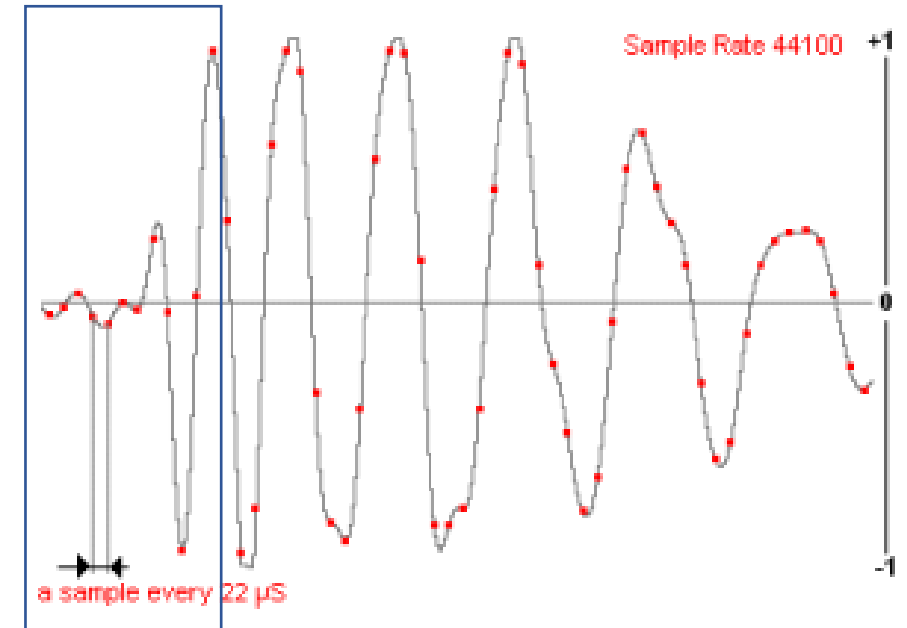
- Audio (songs you listen, my voice...) is a 1D signal
- Stored and transmitted as a series of numbers: [20, 5, 3, 8, 17,...]
- In uncompressed form, the numbers are proportional to sound intensity & voltage of mic/speaker in known intervals.

Filtering

- We often need to extract, remove or boost different frequency ranges (bass, treble, midtones)
- Inside your mics, headphones & guitar amplifiers, this is done in custom chips, that apply various filters

FIR Filters

- Finite Impulse Response (FIR) filters are series of numbers multiplied & added with the sound signal for filtering.
- Numbers are mathematically computed – entire field of DSP



[-2,-1, 2,-2, -3, 0,-1, 7,-1,-23,1,23] – samples: $x[n]$
[0, 1, 2, 5, 2, 1, 0] – filter : $k[n]$

Dot product:

[0,-1, 4,-10,-6, 0, 0] – multiplied: $m[n]$
[-13] – added : $y[0]$

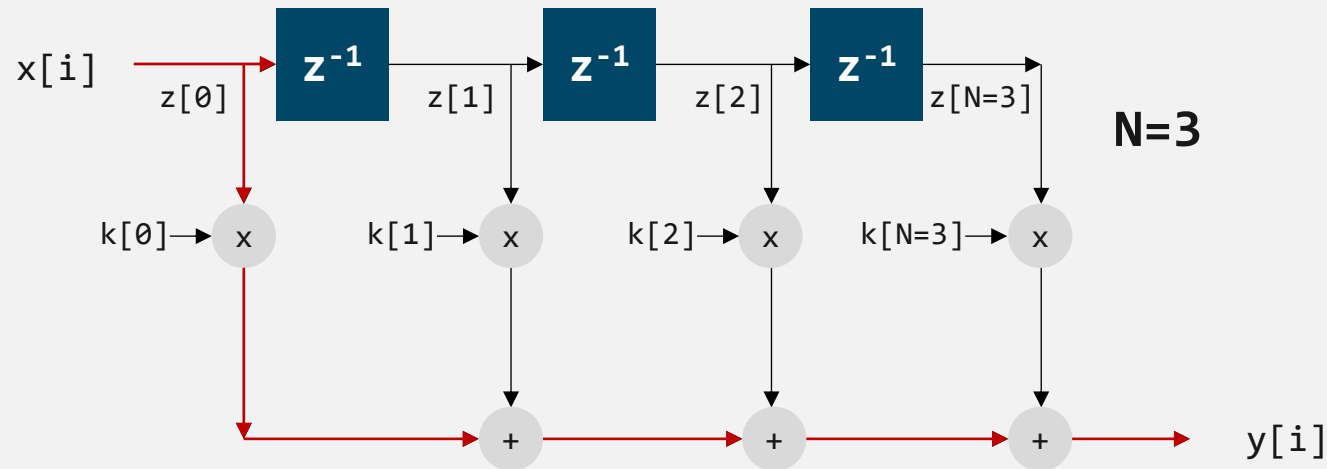
Shift $x[n]$ & dot product:

[-2,-1, 2,-2, -3, 0,-1, 7,-1,-23,1,23] – samples: $x[n]$
[0, 1, 2, 5, 2, 1, 0] – filter : $k[n]$
[0, 2,-4,-15, 0,-1, 0] – multiplied: $m[n]$
[-18] – added : $y[1]$

Repeat shift $x[n]$ & dot product: Convolution

5. FIR Filter Circuit

Abarajithan, 2025
abarajithan07@gmail.com



$$y[n] = x[n]k[0] + x[n-1]k[1] + x[n-2]k[2] + \dots + x[n-N]k[N]$$

$$y[n] = x[n]k[0] + z^{-1}x[n]k[1] + z^{-2}x[n]k[2] + \dots + z^{-N}x[n]k[N]$$

Implementation:

- Every clock cycle, an input sample goes in, output sample comes out
- $N+1$ multipliers and N adders work in parallel for dot product
- $F_{max} = 476$ MHz for $N=64$
- Long path through adders.
- This example is from [Chisel talks](#). Unlike they say, (System)Verilog is highly parametrizable, and we do this all the time.

Fully parametrized!

```

module fir_filter #(
    parameter N = 5, W_X = 8, W_K = 3,
    parameter logic signed [W_K-1:0] K [N+1] = {
        1, 2, 3, 4, 5, 6}, // '{default:0}',

    localparam W_Y = W_X + W_K + $clog2(N+1)
)(
    input  clk, rstn,
    input  logic signed [W_X-1:0] x,
    output logic signed [W_Y-1:0] y
);
    genvar n;
    logic signed [N:0][W_X-1:0] z;

    assign z[0] = x;
    always_ff @(posedge clk or negedge rstn)
        z[N:1] <= !rstn ? '0 : z[N-1:0];

    always_comb begin
        y = 0;
        for (int n=0; n < N+1; n=n+1)
            y = y + K[n] * $signed(z[n]);
        end
endmodule
    
```

Multiplication: data widths get summed
Addition: Each adder gives one extra bit

N shift registers

N+1 multipliers
N adders

5. FIR Filter – Simulation, File I/O

```
module fir_filter_tb;
    timeunit 1ns/1ps;
    localparam W_X = 4,
               W_K = 4,
               N = 3,
               W_Y = W_X + W_K + $clog2(N+1);
```

```
localparam logic signed [W_K-1:0] K [N+1] = {1, 2, 3, 4};
```

```
logic clk=0, rstn=0;
localparam CLK_PERIOD = 10;
initial forever #(CLK_PERIOD/2) clk <= ~clk;
```

```
logic signed [W_X-1:0] x=0;
logic signed [W_Y-1:0] y;
```

```
fir_filter #(.N(N), .W_X(W_X), .W_K (W_K), .K(K)) dut (.*);
```

```
logic signed [W_X-1:0] zi [N+1] = '{default:0};
logic signed [W_X-1:0] zq [$] = zi;
```

```
int status, y_exp=0;
```

```
int file_x = $fopen("D:/x.txt", "r");
int file_y = $fopen("D:/y.txt", "w");
```

File descriptors are int
Read write modes

Parameter array of coefficients

Clock generation

Parametrized Instantiation

Unpacked array – fixed size

Queue – dynamic list

```
// Drive signals
initial begin
    $dumpfile("dump.vcd"); $dumpvars(0, dut);
```

```
#10 rstn <= 1;
```

```
while (!$feof(file_x))
    @(posedge clk) #1 status = $fscanf(file_x, "%d\r", x);
    $fclose(file_x);
```

```
$fclose(file_y);
$finish();
```

Remember to
close the
output file

Reading a file:

- eof – end of file
- fscanf – parses line by line using given format into given variable (x)
- fclose – after reading

```
// Monitor signals
initial forever begin
    @(posedge clk) #2
    zq.push_front(x);
    zq.pop_back();
```

Appending at the beginning of a queue
Remove the last element

```
y_exp = 0;
foreach (zq[i])
    y_exp += zq[i]*K[i];
```

Foreach loop. Note: i is auto inferred

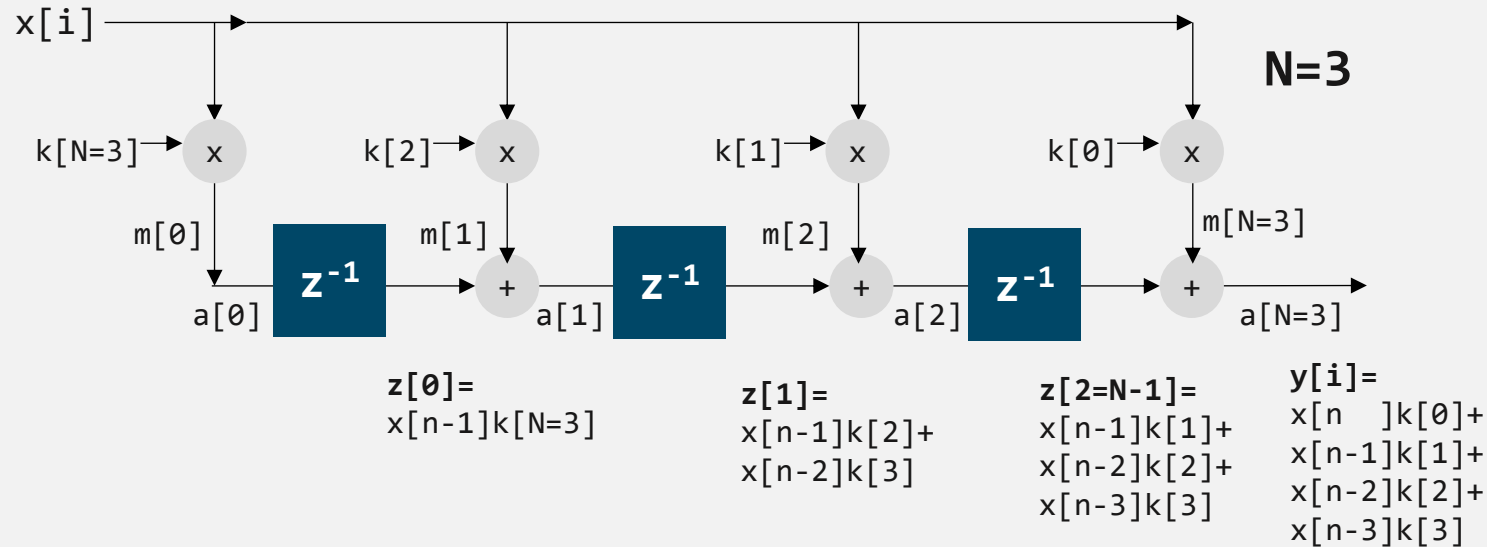
```
assert (y==y_exp) begin
    $display("OK: y:%d", y);
    $fdisplay(file_y, "%d", y);
end else $display("Error: y:%d != y_exp:%d", y, y_exp);
```

Display on screen
Write into "y.txt"

```
end
endmodule
```

5. FIR Filter Circuit - Retiming

Abarajithan, 2025
abarajithan07@gmail.com



$$y[n] = x[n]k[0] + x[n-1]k[1] + x[n-2]k[2] + \dots + x[n-N]k[N]$$

$$y[n] = x[n]k[0] + z^{-1}(x[n]k[1] + z^{-1}(x[n]k[2] + \dots + z^{-1}(x[n]k[N])))$$

Implementation:

- Delay is moved to break long combinational path
- Filter (k) order is reversed
- Math is same
- Fmax = 1 GHz for N=64 (vs 476MHz = 2.1x improvement)

```
module fir_filter_2 #(
    parameter N = 5, W_X = 8, W_K = 3,
    parameter logic signed [W_K-1:0] K [N+1] = {
        8'd1, 8'd2, 8'd3, 8'd4, 8'd5, 8'd6}, // '{default:0}',

    localparam W_M = W_X + W_K, W_Y = W_M + $clog2(N)
)(
    input clk, rstn,
    input logic signed [W_X-1:0] x,
    output logic signed [W_Y-1:0] y
);
```

Same interface & behaviour

```
genvar n;
logic signed [N :0][W_M-1:0] m;
logic signed [N :0][W_Y-1:0] a;
logic signed [N-1:0][W_Y-1:0] z;
```

```
for (n=0; n<N+1; n=n+1)
    assign m[n] = x * K[N-n];
```

```
assign a[0] = m[0];
for (n=1; n<N+1; n=n+1)
    assign a[n] = m[n] + z[n-1];
```

```
for (n=0; n<N; n=n+1)
    always_ff @(posedge clk or negedge rstn)
        z[n] <= !rstn ? '0 : a[n];
```

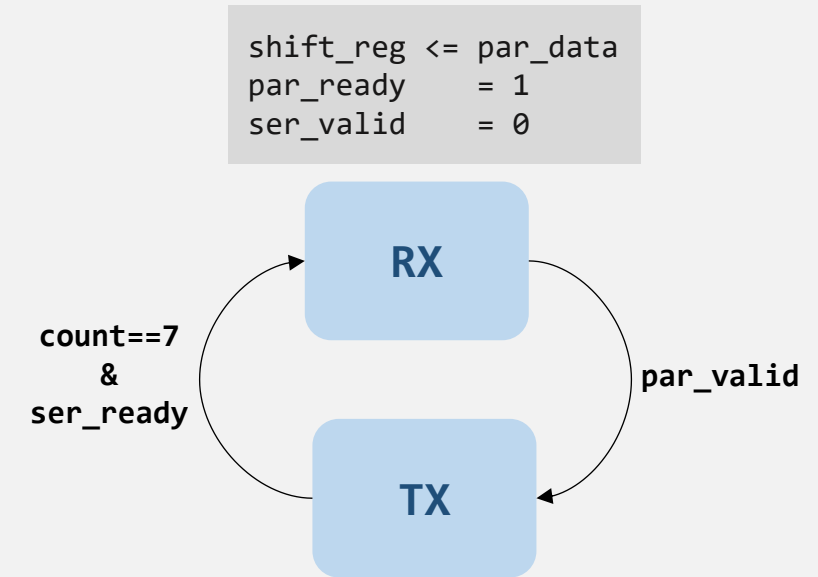
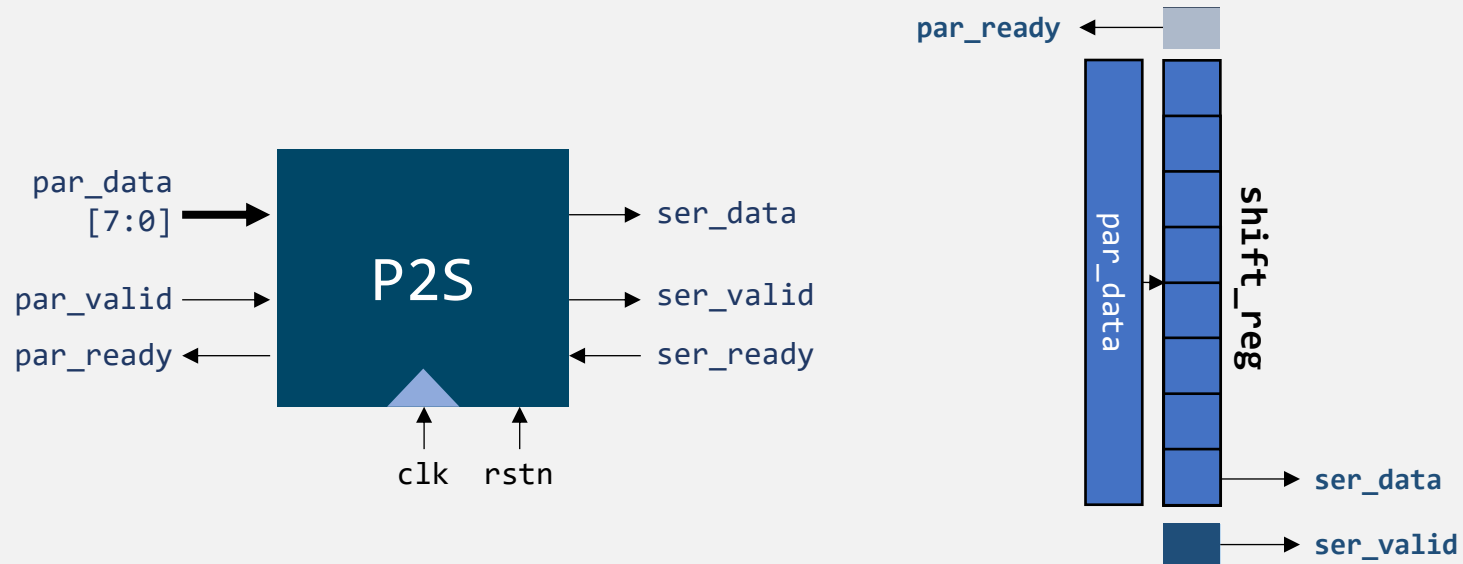
```
assign y = a[N];
endmodule
```

N+1 Multipliers

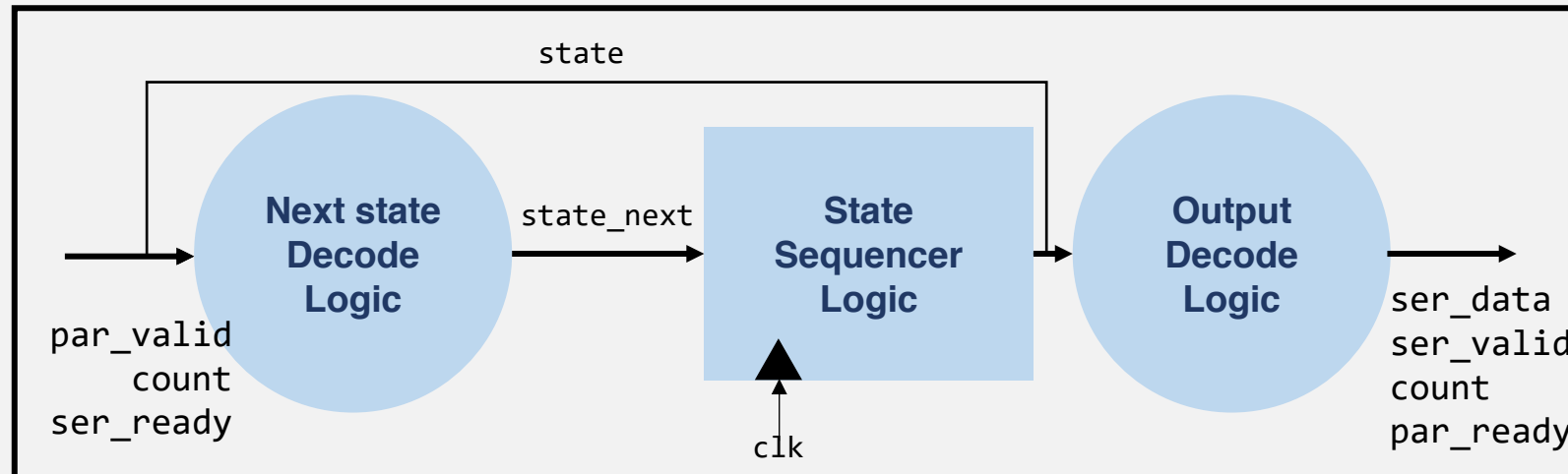
N adders

N registers

7. AXI Stream Parallel to Serial Converter



STATE MACHINE: THREE-PROCEDURE CODING STYLE



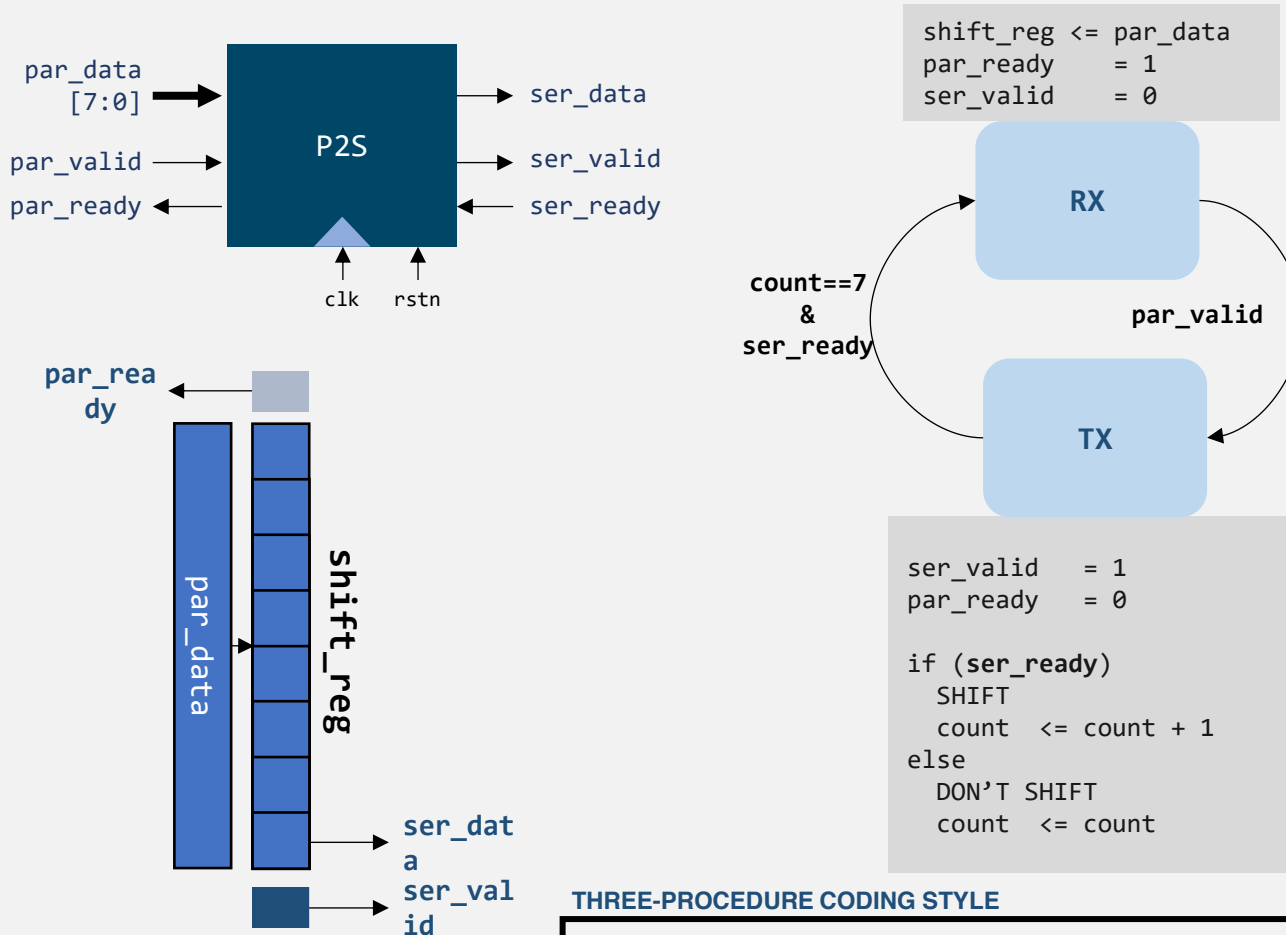
```

ser_valid  = 1
par_ready  = 0

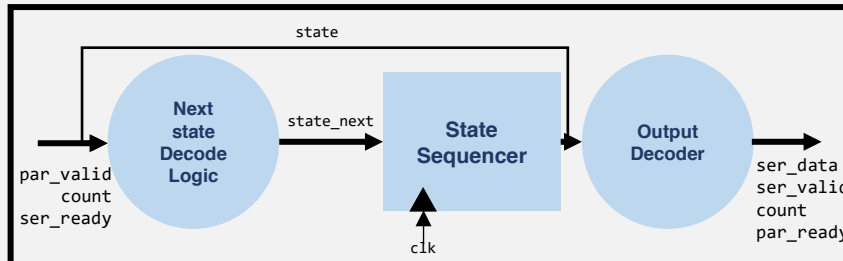
if (ser_ready)
  SHIFT
  count <= count + 1
else
  DON'T SHIFT
  count <= count
    
```

7. AXIS Parallel to Serial Converter

Abarajithan, 2025
abarajithan07@gmail.com



THREE-PROCEDURE CODING STYLE



```
module p2s #(N = 8)(
    input  logic clk, rstn, ser_ready, par_valid,
    input  logic [N-1:0] par_data,
    output logic par_ready, ser_data, ser_valid
);
    localparam N_BITS = $clog2(N);
    enum logic {RX=0, TX=1} next_state, state; // = RX;
    logic [N_BITS-1:0] count; // = 0;
    logic [N-1:0] shift_reg;
```

```
always_comb unique case (state)                                NEXT STATE DECODER
    RX: next_state = par_valid                                ? TX : RX;
    TX: next_state = ser_ready && count==N-1 ? RX : TX;
endcase
```

```
always_ff @(posedge clk or negedge rstn)                      STATE
    state <= !rstn ? RX : next_state;                          SEQUENCER
```

```
assign ser_data  = shift_reg[0];                                OUTPUT DECODER
assign par_ready = (state == RX);
assign ser_valid = (state == TX);
```

```
always_ff @(posedge clk or negedge rstn)
    if (!rstn) count <= '0;
    else unique case (state)
        RX: begin
            shift_reg <= par_data;
            count <= '0;
        end
        TX: if (ser_ready) begin
            shift_reg <= shift_reg >> 1;
            count <= count + 1'd1;
        end
    endcase
```

endmodule

If slave (downstream) gives ready and master (upstream) gives valid in the same clock, data is transferred.

7. Simulation

Abarajithan, 2025
abarajithan07@gmail.com

```
`timescale 1ns/1ps
```

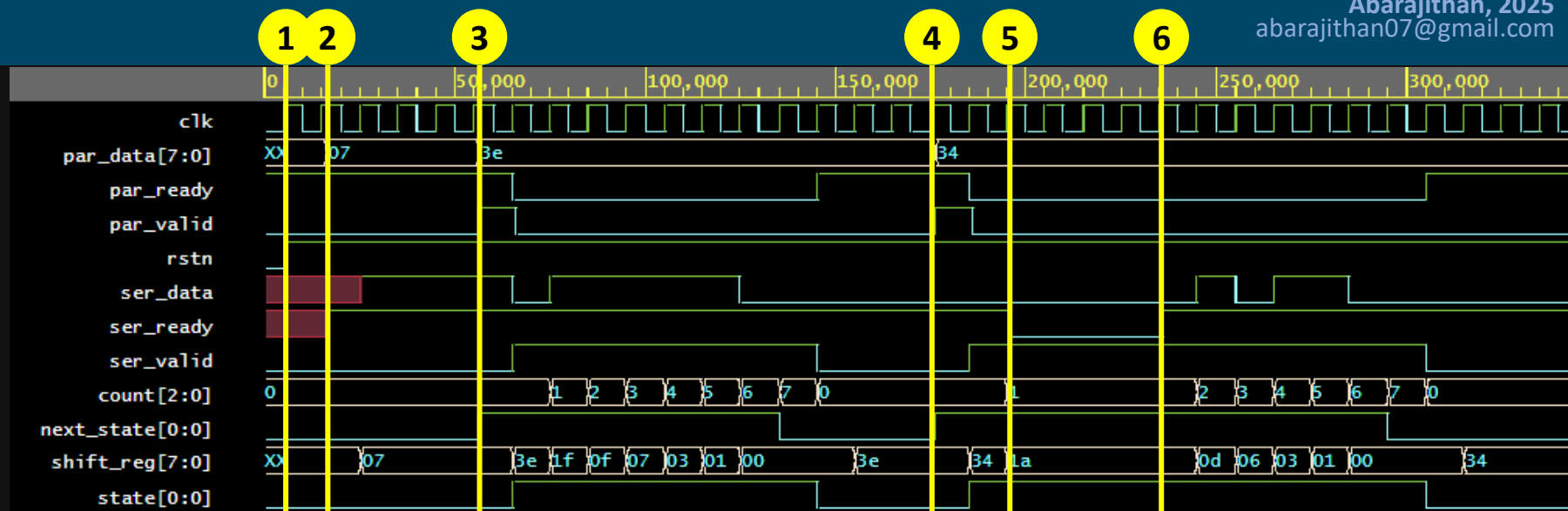
```
module p2s_tb;
```

```
    logic clk = 0, rstn = 0;
    localparam CLK_PERIOD = 10;
    initial forever
        #(CLK_PERIOD/2)
        clk <= ~clk;
```

```
    parameter N = 8;
    logic [N-1:0] par_data;
    logic par_valid=0, par_ready,
        ser_data, ser_valid,
        ser_ready;
```

```
    p2s #(.N(N)) dut (.*);
```

```
    initial begin
        $dumpfile("dump.vcd");
        $dumpvars;
        @(posedge clk); #1 rstn <= 0;
        1 @(posedge clk); #1 rstn <= 1;
```



2 @(posedge clk) #1 par_data <= 8'd7 ; par_valid <= 0; ser_ready <= 1;
#(CLK_PERIOD*3)

3 @(posedge clk) #1 par_data <= 8'd62; par_valid <= 1;
1, ALWAYS assign after a delay from
@(posedge clk) #1 par_valid <= 0; posedge clk. Else weird behavior
#(CLK_PERIOD*10)

4 @(posedge clk) #1 par_data <= 8'd52; par_valid <= 1;
@(posedge clk) #1 par_valid <= 0;

5 @(posedge clk) #1 ser_ready <= 0;
#(CLK_PERIOD*3)

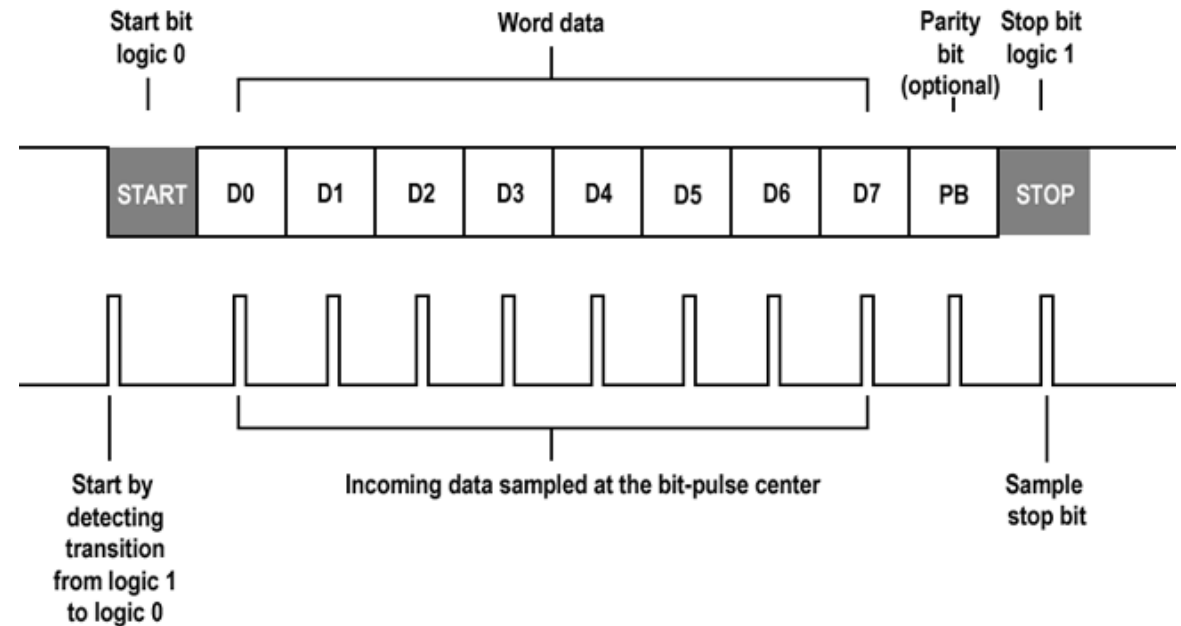
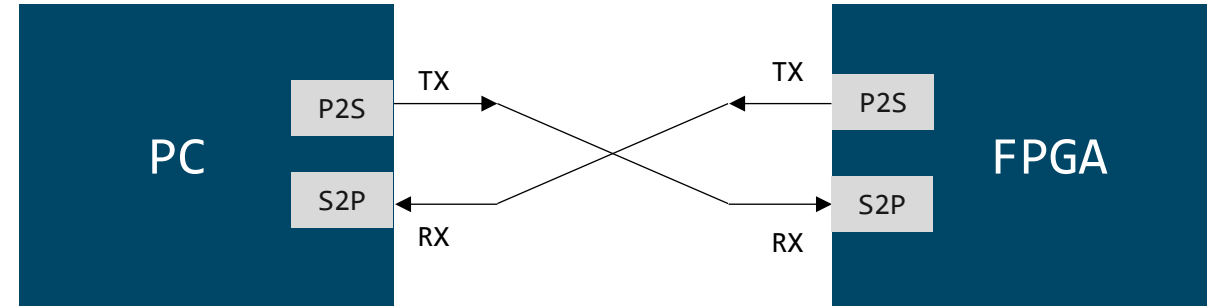
6 @(posedge clk) #1 ser_ready <= 1;
#(CLK_PERIOD*10)
@(posedge clk) #1 ser_ready <= 0;
\$finish();

```
    end  
endmodule
```


UART Serial Bus

Abarajithan, 2025
abarajithan07@gmail.com

- Serial
- Asynchronous
 - Baud rate. eg: 9600 bits per second
 - FPGA (200 MHz): 20,833.33 clocks per bit
- Slow
- Used in microcontrollers



8. UART RX to AXI Stream

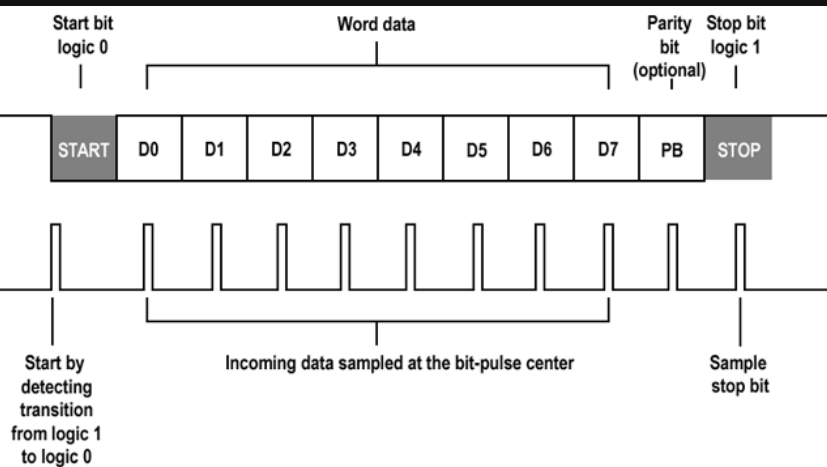
Abarajithan, 2025
abarajithan07@gmail.com

```
module uart_rx #(
    parameter CLOCKS_PER_PULSE = 4, //200_000_000/9600
            BITS_PER_WORD      = 8,
            W_OUT = 16 //R*C*W_K + C*W_X
) (
    input  logic clk, rstn, rx,
    output logic m_valid,
    output logic [W_OUT-1:0] m_data
);
    localparam NUM_WORDS = W_OUT/BITS_PER_WORD;
    enum {IDLE, START, DATA, END} state;
```

// Counters

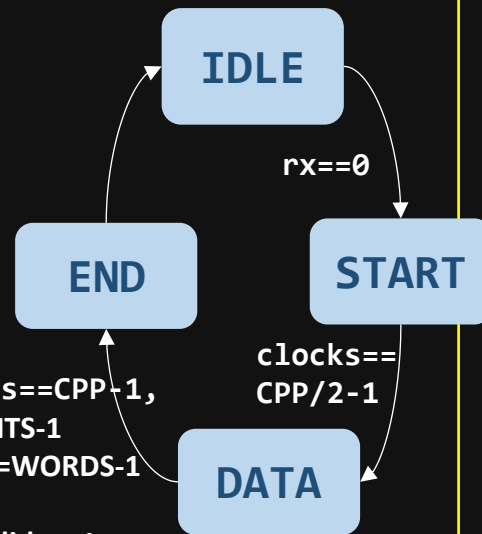
Three counters

```
logic [$clog2(CLOCKS_PER_PULSE) -1:0] c_clocks;
logic [$clog2(BITS_PER_WORD) -1:0] c_bits ;
logic [$clog2(NUM_WORDS) -1:0] c_words ;
```



clocks==CPP-1,
bits=BITS-1
words=WORDS-1

DO: valid <= 1



Count clocks,
bits, words
shift {rx,m_data}
after each bit

```
// State Machine: One procedure coding style
always_ff @(posedge clk or negedge rstn) begin
    if (!rstn) begin
        {c_words, c_bits, c_clocks, m_valid, m_data} <= '0;
        state <= IDLE;
    end else begin
        m_valid <= 0;
        case (state)
            IDLE : if (rx == 0)
                    state <= START;
            START: if (c_clocks == CLOCKS_PER_PULSE/2-1) begin
                    state <= DATA;
                    c_clocks <= 0;
                end else
                    c_clocks <= c_clocks + 1;

            DATA : if (c_clocks == CLOCKS_PER_PULSE-1) begin
                    c_clocks <= 0;
                    m_data <= {rx, m_data[W_OUT-1:1]};

                    if (c_bits == BITS_PER_WORD-1) begin
                        state <= END;
                        c_bits <= 0;

                        if (c_words == NUM_WORDS-1) begin
                            m_valid <= 1;
                            c_words <= 0;

                            end else c_words <= c_words + 1;
                            end else c_bits <= c_bits + 1;
                            end else c_clocks <= c_clocks + 1;

            END : if (c_clocks == CLOCKS_PER_PULSE-1) begin
                    state <= IDLE;
                    c_clocks <= 0;
                end else
                    c_clocks <= c_clocks + 1;
        endcase
    end
end
endmodule
```

8. UART RX Testbench

Abarajithan, 2025
abarajithan07@gmail.com

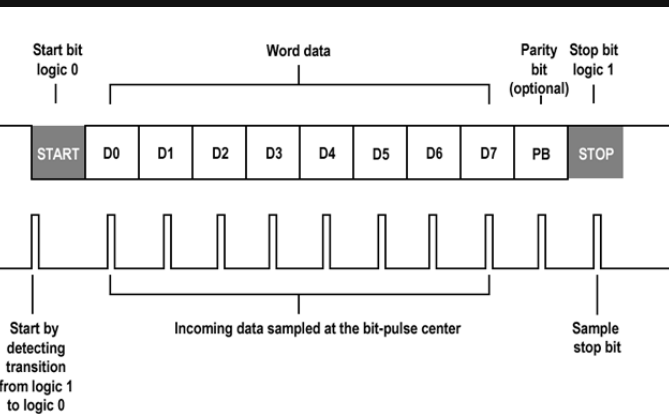
```
module uart_rx_tb;
```

```
    timeunit 1ns/1ps;
    localparam CLOCKS_PER_PULSE = 4, //200_000_000/9600
               W_OUT              = 16,
               BITS_PER_WORD      = 8,
               NUM_WORDS          = W_OUT/BITS_PER_WORD,
               CLK_PERIOD         = 10;
```

```
    logic clk=0, rstn=0, rx=1, m_valid;
    logic [NUM_WORDS-1:0][BITS_PER_WORD-1:0] m_data, data;
    logic [BITS_PER_WORD+2-1:0] packet;
```

```
    initial forever #(CLK_PERIOD/2) clk <= !clk;
```

```
    uart_rx #(
        .CLOCKS_PER_PULSE(CLOCKS_PER_PULSE),
        .W_OUT(W_OUT),
        .BITS_PER_WORD(BITS_PER_WORD)) dut (.*);
```



```
// Driver
```

```
initial begin
```

```
    $dumpfile("dump.vcd"); $dumpvars;
    repeat(2) @(posedge clk) #1;
    rstn = 1;
    repeat(5) @(posedge clk) #1;
```

```
repeat (10) begin
```

```
    data = $urandom(); Randomize data
```

```
    for (int iw=0; iw<NUM_WORDS; iw=iw+1) begin
        packet = {1'b1, data[iw], 1'b0};
```

```
        repeat ($urandom_range(1,20)) @(posedge clk); Random delay
        between packets
```

```
        for (int ib=0; ib<BITS_PER_WORD+2; ib=ib+1)
```

```
            repeat(CLOCKS_PER_PULSE) begin
                #1 rx <= packet[ib];
                @(posedge clk);
            end Send data, hold every bit
            for 4 clocks
```

```
        end
```

```
        repeat ($urandom_range(1,100)) @(posedge clk); Random delay
        between sets
```

```
    end
```

```
    $finish();
```

```
end
```

```
//Monitor
```

```
initial forever @(posedge clk)
```

```
    if(m_valid) assert (m_data == data) $display("OK, %b", m_data);
    else $error("Sent %b, got %b", data, m_data); Parallel monitor
```

```
// Count UART bits on waveform
```

```
int bits;
```

```
initial forever begin
```

```
    bits = 0;
```

```
    wait(!rx);
```

```
    for (int n=0; n<BITS_PER_WORD+2; n++) begin
```

```
        bits += 1; repeat (CLOCKS_PER_PULSE) @(posedge clk);
```

```
    end end
```

```
endmodule
```

Count UART bits to be
able to read waveform

8. AXI Stream to UART TX

Abarajithan, 2025
abarajithan07@gmail.com

```
module uart_tx #(
    parameter CLOCKS_PER_PULSE = 4, //200_000_000/9600
              BITS_PER_WORD     = 8,
              PACKET_SIZE       = BITS_PER_WORD+5,
              W_OUT = 16,

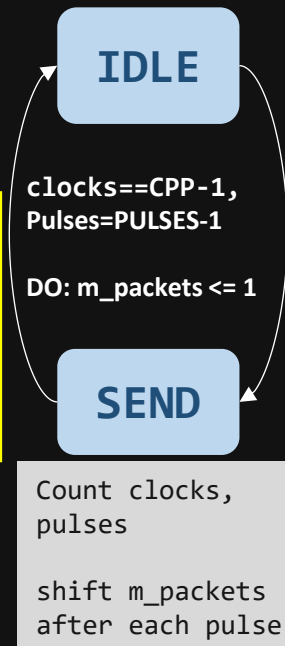
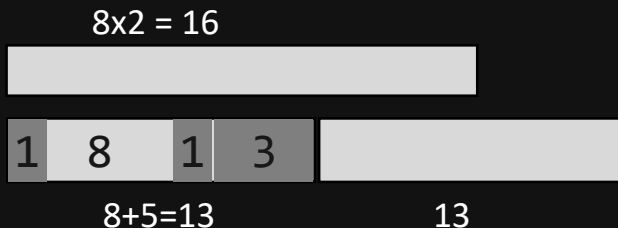
    localparam NUM_WORDS      = W_OUT/BITS_PER_WORD
) (
    input logic clk, rstn, s_valid,
    input logic [NUM_WORDS-1:0][BITS_PER_WORD-1:0] s_data,
    output logic tx, s_ready

);
    localparam END_BITS = PACKET_SIZE-BITS_PER_WORD-1;
    logic [NUM_WORDS-1:0][PACKET_SIZE-1:0] s_packets;
    logic [NUM_WORDS*PACKET_SIZE-1:0] m_packets;
```

Create set of packets. Each with start bit (0), data word and END_BITS number of ones (zeros inverted)

```
genvar n;
for (n=0; n<NUM_WORDS; n=n+1)
    assign s_packets[n] = { ~(END_BITS'(0)), s_data[n], 1'b0};

assign tx = m_packets[0];
```



```
// Counters
logic [$clog2(NUM_WORDS*PACKET_SIZE)-1:0] c_pulses;
logic [$clog2(CLOCKS_PER_PULSE)-1:0] c_clocks;
```

Two nested counters

```
// State Machine
```

```
enum {IDLE, SEND} state;
```

```
always_ff @(posedge clk or negedge rstn) begin
```

```
    if (!rstn) begin
        state <= IDLE;
        m_packets <= '1;
        {c_pulses, c_clocks} <= 0;
    end else
```

```
    case (state)
```

```
        IDLE : if (s_valid) begin
                    state <= SEND;
                    m_packets <= s_packets;
                end
```

Receive data via AXI Stream

```
        SEND : if (c_clocks == CLOCKS_PER_PULSE-1) begin
                    c_clocks <= 0;
```

```
                    if (c_pulses == NUM_WORDS*PACKET_SIZE-1) begin
                        c_pulses <= 0;
                        m_packets <= '1;
                        state <= IDLE;
                    end else begin
                        c_pulses <= c_pulses + 1;
                        m_packets <= (m_packets >> 1);
                    end
```

Keep each bit stable for 4 clocks
Send all 26 bits

```
        end else c_clocks <= c_clocks + 1;
    endcase
```

```
end
```

```
    assign s_ready = (state == IDLE);
```

Receive data via AXI Stream

```
endmodule
```

8. UART TX Testbench

Abarajithan, 2025
abarajithan07@gmail.com

```
module uart_tx_tb;
    timeunit 1ns/1ps;
    localparam CLOCKS_PER_PULSE = 4, //200_000_000/9600
               W_OUT              = 16, BITS_PER_WORD    = 8,
               PACKET_SIZE        = BITS_PER_WORD+5,
               NUM_WORDS          = W_OUT/BITS_PER_WORD,
               CLK_PERIOD         = 10;

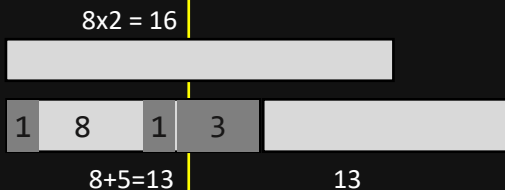
    logic clk=0, rstn=0, tx, s_valid=0, s_ready;
    logic [NUM_WORDS-1:0][BITS_PER_WORD-1:0] s_data, rx_data;
    logic [BITS_PER_WORD-1:0] rx_word;
    initial forever #(CLK_PERIOD/2) clk <= !clk;

    uart_tx #(
        .CLOCKS_PER_PULSE(CLOCKS_PER_PULSE), .BITS_PER_WORD(BITS_PER_WORD),
        .PACKET_SIZE(PACKET_SIZE), .W_OUT(W_OUT)) dut (.*);
```

```
// AXIS Driver
initial begin
    $dumpfile("dump.vcd"); $dumpvars;
    #20 rstn = 1; repeat(5) @(posedge clk) #1;

    repeat (10) begin
        repeat ($urandom_range(1,20)) @(posedge clk);
        wait (s_ready);

        @(posedge clk) #1 s_data = $urandom(); s_valid = 1;
        @(posedge clk) #1 s_valid = 0;
        wait (s_ready);
    end
    $finish();
end
```



AXIS Driver

```
// Monitor
initial forever begin
```

```
    rx_data <= 'x;
    for (int iw=0; iw<NUM_WORDS; iw=iw+1) begin
        wait(!tx);
        // go to middle of start bit
        repeat (CLOCKS_PER_PULSE/2) @(posedge clk);
        for (int ib=0; ib<BITS_PER_WORD; ib=ib+1) begin
            // go to middle of data bit
            repeat (CLOCKS_PER_PULSE) @(posedge clk);
            rx_word[ib] = tx;
        end
        rx_data[iw] = rx_word;
        for (int ib=0; ib<PACKET_SIZE-BITS_PER_WORD-1; ib=ib+1) begin
            repeat (CLOCKS_PER_PULSE) @(posedge clk);
            assert (tx == 1) else $error("Incorrect end bits/padding");
        end
    end
```

For each word

Start

Data

End

```
    assert (rx_data == s_data) $display("OK, %b", rx_data);
    else $error("Sent %b, got %b", s_data, rx_data);
end
```

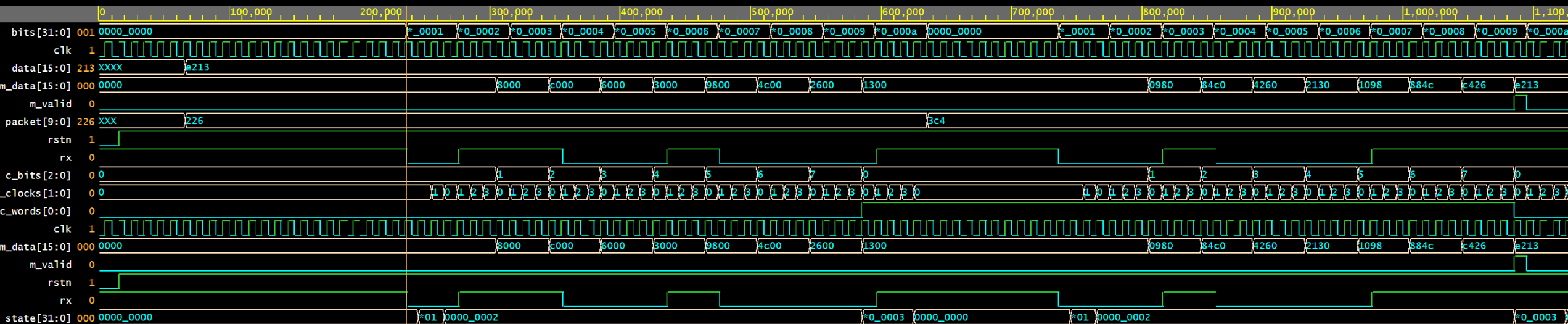
Check

```
// Count UART bits on waveform
```

```
int bits;
initial forever begin
    bits = 0;
    wait(!tx);
    for (int n=0; n<PACKET_SIZE; n++) begin
        bits += 1;
        repeat (CLOCKS_PER_PULSE) @(posedge clk);
    end
end
```

Count UART bits to be
able to read waveform

endmodule

RX

TX



AXI Stream Bus

Bus

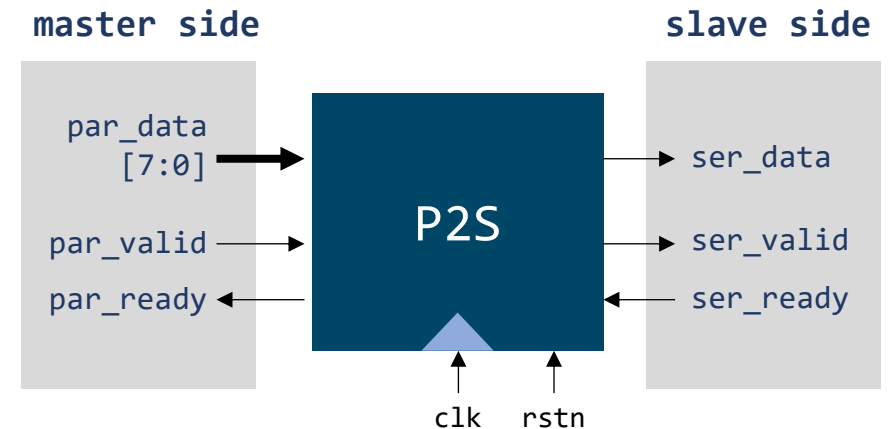
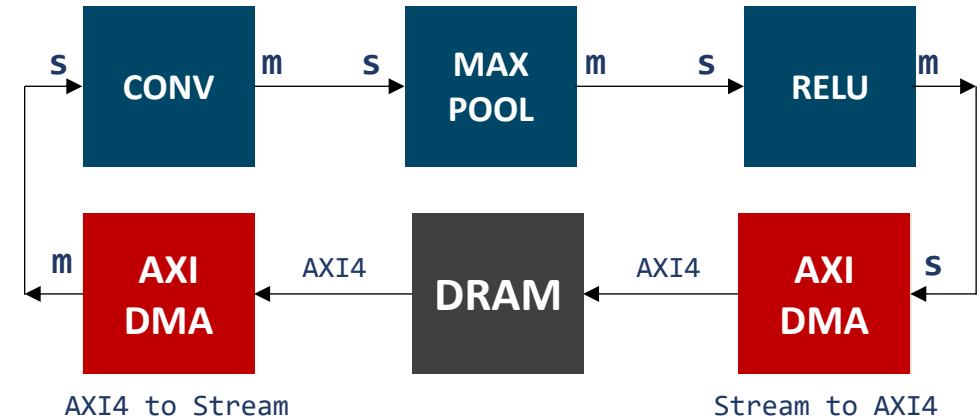
- Standardized set of wires and a “protocol”
- Used to keep modules independent & reusable when designing anything larger than this
- Eg: Bluetooth, Wi-Fi...etc. of mobile system-on-chips are made by different companies. But they follow the AXI (full) bus protocol.

AXI Stream

Wires : data, valid, [ready, last, user, id...]

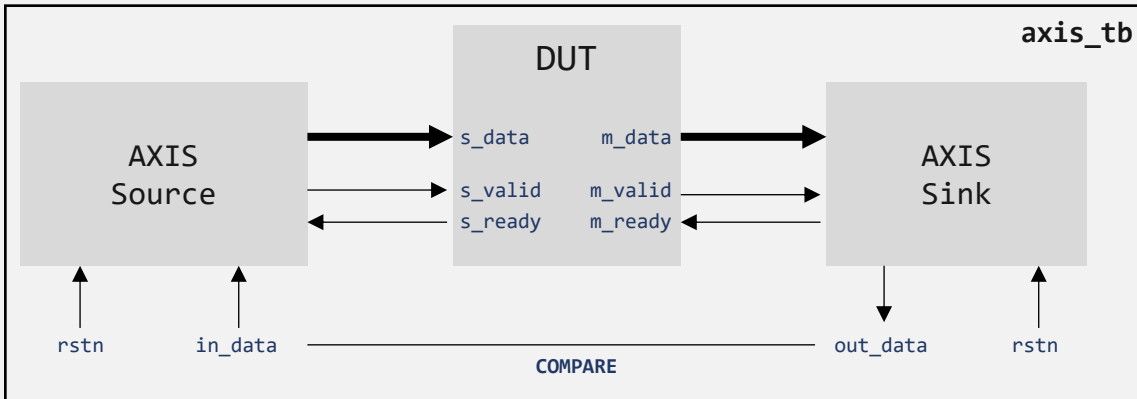
Protocol : If slave (downstream) gives ready and master (upstream) gives valid in the same clock, data is transferred.

- Simplest & fastest bus I’ve seen. From ARM (AMBA/AXI)
- Useful for data pipelines where modules have independent data rates
- Backpressure: when a slave gets busy & pulls down ready
- Can be chained without long paths (unlike ours) using a skid buffer
- Each full AXI bus is made of 5 AXI Stream buses



Verifying AXI Stream Behavior of Your Modules

Abarajithan, 2025
abarajithan07@gmail.com



- Sink
 - Simulates the slave that pulls data
 - Randomly toggle ready with a probability
- Source
 - Simulates the master that pushes data
 - Randomly toggle valid with a probability
 - Scramble data when invalid
- Compare data in top level testbench
- Simple AXIS (ready, valid, data). For full (keep, last), see:
github.com/abarajithan11/dnn-engine/blob/master/test/sv/axis_tb.sv

```
module AXIS_Sink #(
    parameter WORD_W=8, BUS_W=8, PROB_READY=20, 20%
               N_BEATS=10,
               WORDS_PER_BEAT = BUS_W/WORD_W
)()
    input  logic clk, m_valid,
    output logic m_ready=0,
    input  logic [WORDS_PER_BEAT-1:0][WORD_W-1:0] m_data
);

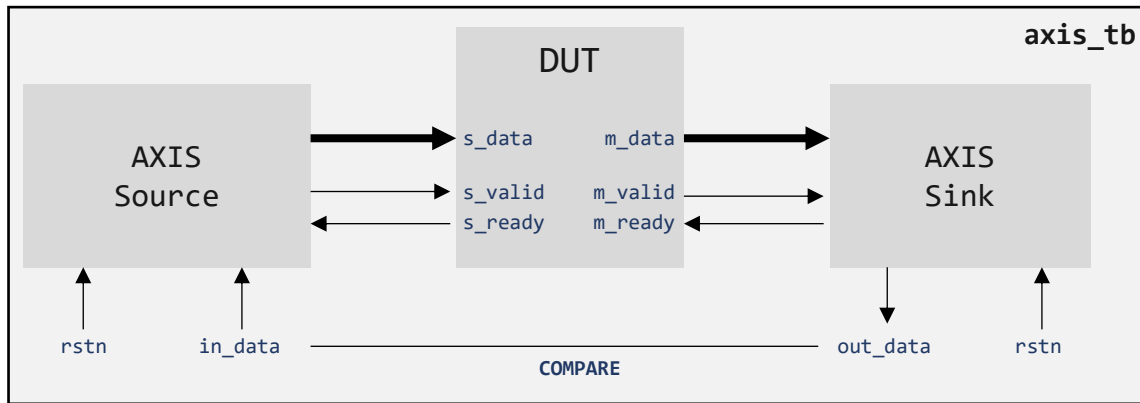
task automatic axis_pull_packet(                                Full data received
    output logic [N_BEATS-1:0][WORDS_PER_BEAT-1:0][WORD_W-1:0] packet
);
    // Loop over beats                                           Only receive a packet (N_BEATS beats) and return
    for (int i=0; i<N_BEATS; i++) begin
        Keep ready=0 with (1-P), ready=1 for one clock, check valid.
        do begin                                                  If valid, copy data, else ready=0 and continue
            m_ready <= 0; // keep m_ready low with prob (1-PROB_READY)
            while ($urandom_range(0,99) >= PROB_READY) @(posedge clk);
            m_ready <= 1;
            @(posedge clk); // keep m_ready high for one cycle
        end while (!m_valid); // if m_valid is high, break out of loop

        packet[i] = m_data; //sample m_data
    end
endtask
endmodule
```

Copy data on handshake

Verifying AXI Stream Behavior of Your Modules...

Abarajithan, 2025
abarajithan07@gmail.com

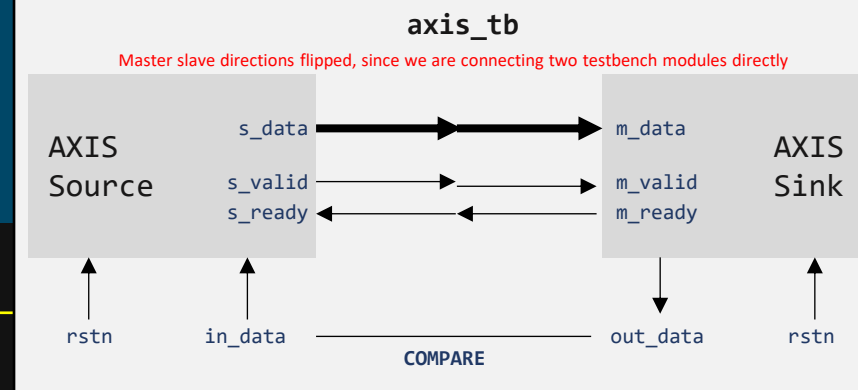


```
module AXIS_Source #(
    parameter WORD_W=8, BUS_W=8, PROB_VALID=20,
               N_BEATS=10,
    localparam WORDS_PER_BEAT = BUS_W/WORD_W
)(
    input  logic clk, s_ready,          Initialize s_valid & s_data
    output logic s_valid=0,
    output logic [WORDS_PER_BEAT-1:0][WORD_W-1:0] s_data = 'x
);
task automatic axis_push_packet(          Full data to send
    input  logic [N_BEATS-1:0][WORDS_PER_BEAT-1:0][WORD_W-1:0] packet
);
    for (int i=0; i<N_BEATS; i++) begin          Keep svalid low with (1-P)
        // randomize s_valid and wait
        while ($urandom_range(0,99) >= PROB_VALID) @(posedge clk);

        s_valid <= 1;          Once svalid is high, set data, wait for ready (protocol)
        s_data <= packet[i];
        // wait for s_data to be accepted
        do @(posedge clk); while (!s_ready);
        // clear s_valid and s_data          Clear valid and data after handshake
        s_valid <= 0;
        s_data <= 'x;
    end
endtask
endmodule
```

Verifying AXI Stream Behavior of Your Modules.....

Abarajithan, 2025
abarajithan07@gmail.com



```

module axis_tb;
    timeunit 1ns/1ps;
    localparam WORD_W=8, BUS_W=8,
               N_BEATS=10, WORDS_PER_BEAT=BUS_W/WORD_W,
               PROB_VALID=10, PROB_READY=10,
               CLK_PERIOD=10, NUM_EXP=500;

    logic clk=0, rstn=1;
    logic s_ready, s_valid, m_ready, m_valid;
    logic [WORDS_PER_BEAT-1:0][WORD_W-1:0] s_data, m_data, in_beat;
    logic [N_BEATS-1:0][WORDS_PER_BEAT-1:0][WORD_W-1:0] in_data, out_data, exp_data;

    logic [N_BEATS*WORD_W*WORDS_PER_BEAT-1:0] queue [$];

    initial forever #(CLK_PERIOD/2) clk <= ~clk;
    
```

```

AXIS_Source #(
    .WORD_W(WORD_W), .BUS_W(BUS_W),
    .PROB_VALID(PROB_VALID), .N_BEATS(N_BEATS)) source (.*)
AXIS_Sink #(
    .WORD_W(WORD_W), .BUS_W(BUS_W),
    .PROB_READY(PROB_READY), .N_BEATS(N_BEATS)) sink (.*)
    
```

```

// skid_buffer #(BUS_W) dut (.*)
assign s_ready = m_ready;
assign m_data = s_data;
assign m_valid = s_valid;
    
```

DRIVER

```

// Driver
initial begin
    $dumpfile ("dump.vcd"); $dumpvars;
    repeat(NUM_EXP) begin
        // Randomize in data
        foreach (in_data[n]) begin
            foreach (in_beat[w])
                in_beat[w] = $urandom_range(0,2**WORD_W-1);
            in_data[n] = in_beat;
        end
        queue.push_front(in_data); // append to end of queue
        source.axis_push_packet;
    end
end
    
```

Push into queue & start task.
Task will return after pushing a packet (N_BEATS)

MONITOR

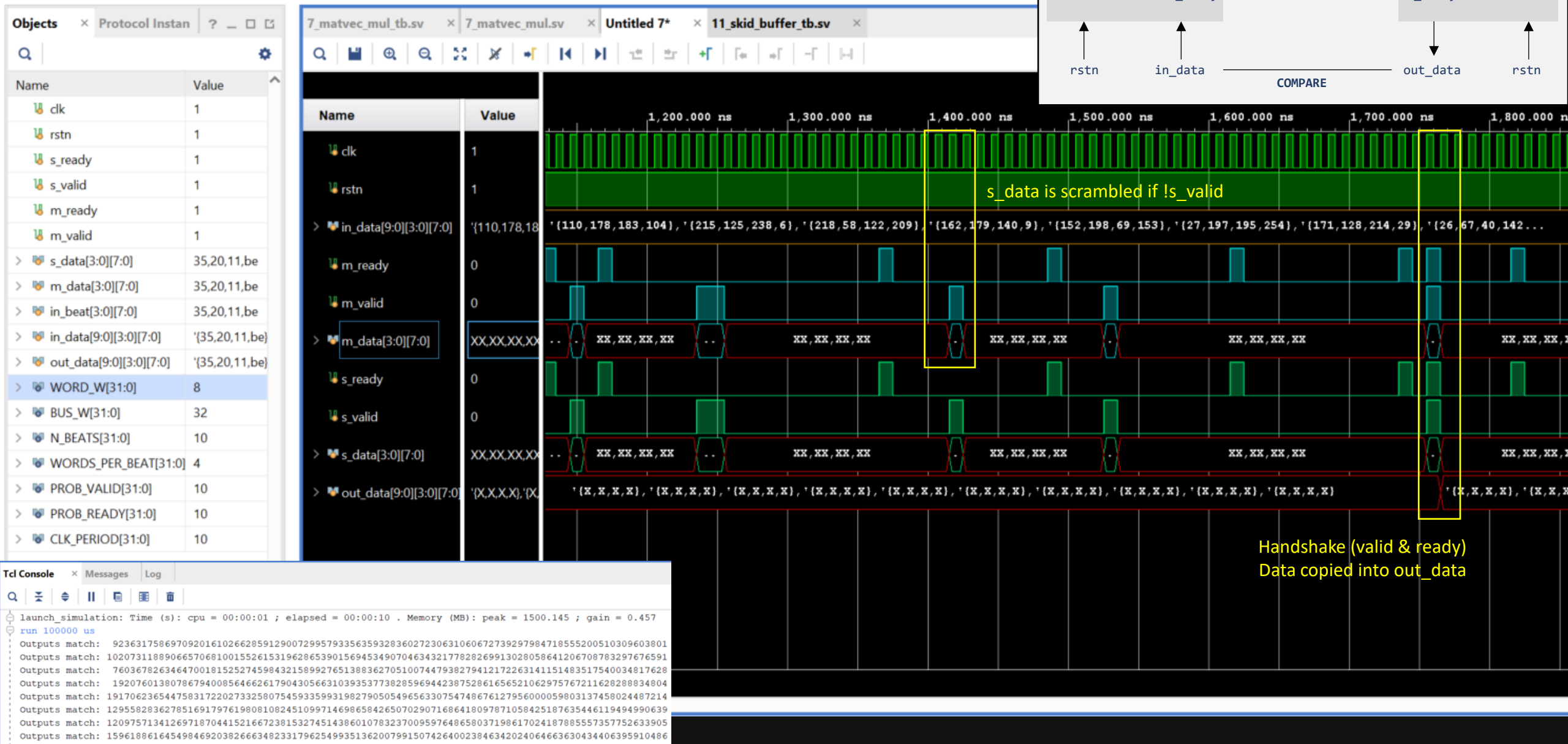
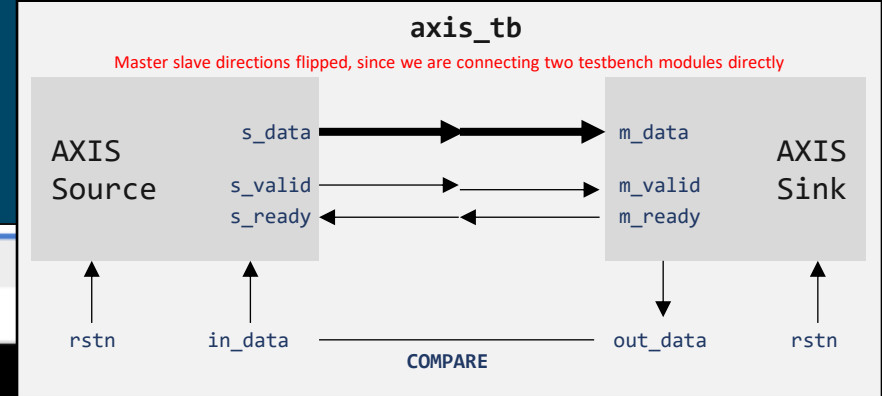
```

// Monitor
initial begin
    repeat(NUM_EXP) begin
        sink.axis_pull_packet;
        exp_data = queue.pop_back();
        assert (exp_data == out_data) // remove last element
            $display("Outputs match: %d", exp_data);
        else $fatal(0, "Expected: %d != Output: %d", exp_data, out_data);
    end
end
$finish();
endmodule
    
```

Start task. Task will return after pulling a packet (N_BEATS)
Compare

Verifying AXI Stream Behavior of Your Modules.....

Abarajithan, 2025
abarajithan07@gmail.com

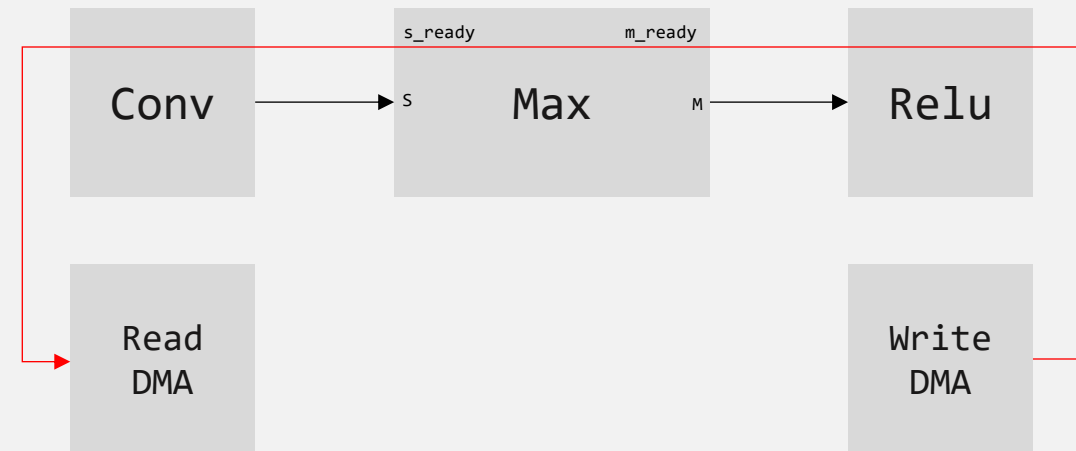
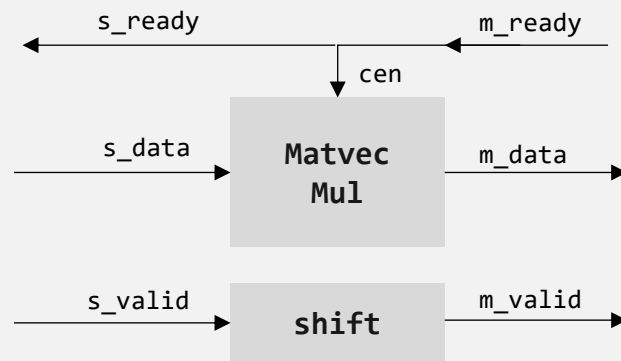


AXI Stream: Combinational ready

- If downstream cannot accept data, and if you want to send data, you need to freeze immediately

```
assign s_ready = !(m_valid & !m_ready);
```

- Causes a critical path in AXI Stream pipelines

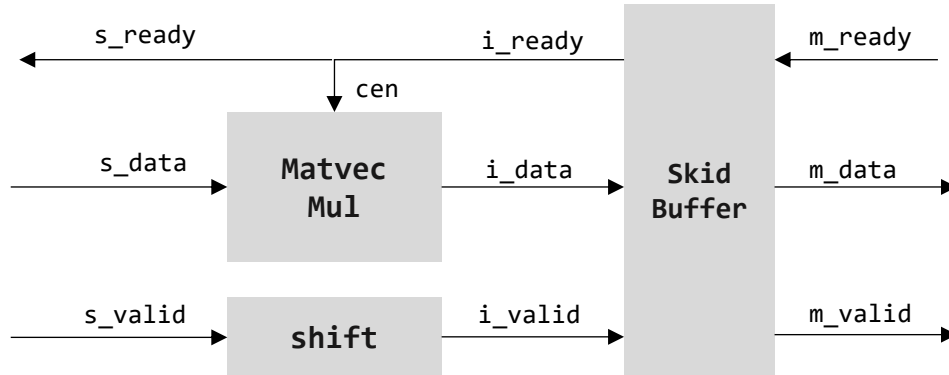


What if we register `m_ready`?

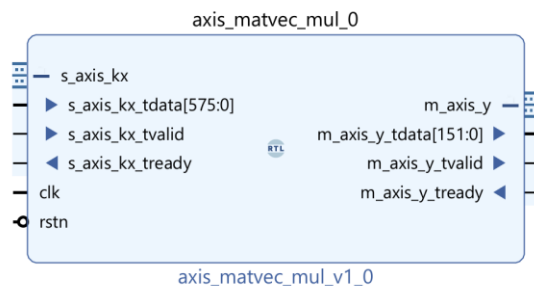
- Our master will know only after a clock cycle
- Master will give us data in this clock, but we cannot send out data in this clock

AXIS MVM + Old Verilog Wrapper

Abarajithan, 2025
abarajithan07@gmail.com



- AXI Stream renamed to `s_axis_NAME_tX`
- Old Verilog: Vivado block design (SOC)
 - reg: output of an always procedure (comb too)
 - No localparam inside parameter declaration



```
module axis_matvec_mul #(
    parameter R=8, C=8, W_X=8, W_K=8,
    LATENCY = $clog2(C)+1, W_Y = W_X + W_K + $clog2(C)) (
    input  clk, rstn,
    output s_axis_kx_tready,
    input  s_axis_kx_tvalid,
    input  [R*C*W_K + C*W_X -1:0] s_axis_kx_tdata,
    input  m_axis_y_tready,
    output m_axis_y_tvalid,
    output [R*W_Y -1:0] m_axis_y_tdata
);
    wire [R*C*W_K-1:0] k;
    wire [C*W_X -1:0] x;
    assign {k, x} = s_axis_kx_tdata;

    wire [R*W_Y-1:0] i_data;
    wire i_ready;

    matvec_mul #(.R(R), .C(C), .W_X(W_X), .W_K(W_K))
        MATVEC (.clk(clk), .cen(i_ready), .k(k), .x(x), .y(i_data));

    reg [LATENCY-2:0] shift;
    reg i_valid;

    always @(posedge clk or negedge rstn)
        if (!rstn) {i_valid, shift} <= 0;
        else if(i_ready) {i_valid, shift} <= {shift, s_axis_kx_tvalid};

    skid_buffer #(.WIDTH(R*W_Y)) SKID (
        .clk      (clk), .rstn (rstn),
        .s_ready  (i_ready),
        .s_valid  (i_valid),
        .s_data   (i_data),
        .m_ready  (m_axis_y_tready),
        .m_valid  (m_axis_y_tvalid),
        .m_data   (m_axis_y_tdata));

    assign s_axis_kx_tready = i_ready;

endmodule
```

No localparams here

No packed arrays

Named like this for vivado block diagram to automatically group

No logic.
Wire – assign, Reg - always

Reg – output of always (comb or ff)

Only one always type

AXIS MVM Testbench

Abarajithan, 2025
abarajithan07@gmail.com

```
module axis_matvec_mul_tb;
  timeunit 1ns/1ps;
  localparam R=8, C=8, W_X=8, W_K=8,
             W_Y = W_X + W_K + $clog2(C),
             BUS_IN_W = R*C*W_K + C*W_X,
             BUS_OUT_W = R*W_Y,
             PROB_VALID=10, PROB_READY=10,
             CLK_PERIOD=10, NUM_EXP=500;

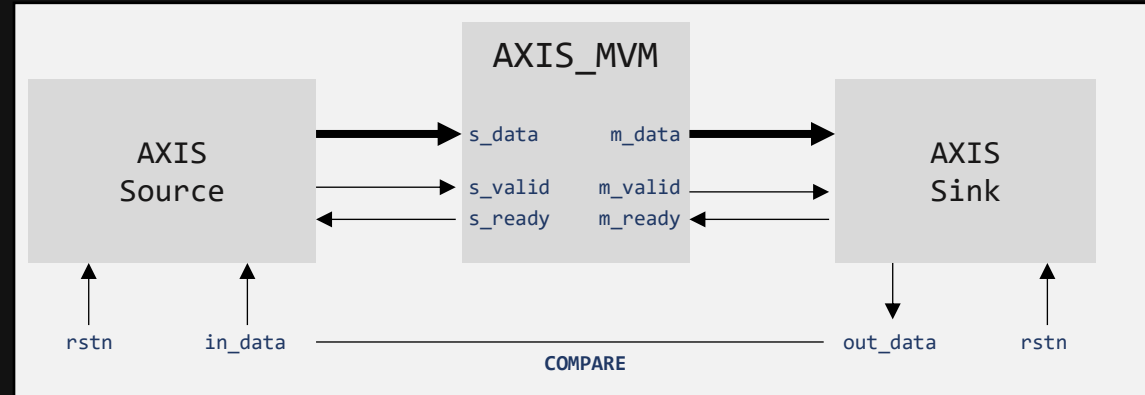
  logic clk=0, rstn=0;
  initial forever #(CLK_PERIOD/2) clk <= ~clk;
```

```
  logic s_ready, s_valid, m_ready, m_valid;
```

```
  logic [C-1:0][W_X-1:0] x_in, x_out;
  logic [R-1:0][C-1:0][W_K-1:0] k_in, k_out;
  logic [C-1:0][W_K-1:0] k_row_in, k_row_out;
  logic [BUS_IN_W -1:0] s_axis_kx_tdata, s_data, in_data;
  logic [R-1:0][W_Y-1:0] m_axis_y_tdata, m_data, out_data, exp_data;
  logic [BUS_IN_W -1:0] queue [$];
```

```
  AXIS_Source #(.WORD_W(BUS_IN_W ), .BUS_W(BUS_IN_W ), .PROB_VALID(PROB_VALID), .N_BEATS(1)) source (.*);
  AXIS_Sink   #(.WORD_W(BUS_OUT_W), .BUS_W(BUS_OUT_W), .PROB_READY(PROB_READY), .N_BEATS(1)) sink   (.*);
```

```
  logic s_axis_kx_tready, s_axis_kx_tvalid, m_axis_y_tready, m_axis_y_tvalid;
  assign {s_ready, s_axis_kx_tvalid, s_axis_kx_tdata} = {s_axis_kx_tready, s_valid, s_data};
  assign {m_axis_y_tready, m_valid, m_data} = {m_ready, m_axis_y_tvalid, m_axis_y_tdata};
  axis_matvec_mul #(.R(R),.C(C),.W_X(W_X),.W_K(W_K)) dut (.*);
```



Source & Sink

AXIS MVM Testbench...

Abarajithan, 2025
abarajithan07@gmail.com

Driver

```
// AXIS Driver
initial begin
    $dumpfile ("dump.vcd"); $dumpvars;

    repeat (2) @ (posedge clk);
    rstn = 1;

    repeat(NUM_EXP) begin
        // Randomize in_data
        foreach (x_in[c])
            x_in[c] = $urandom_range(0,2**W_X-1);
        foreach (k_in[r]) begin
            foreach (k_row_in[c])
                k_row_in[c] = $urandom_range(0,2**W_K-1);
            k_in[r] = k_row_in;
        end
        in_data = {k_in, x_in};

        // append to end of queue
        queue.push_front(in_data);
        #1;
        source.axis_push_packet(in_data);
    end
end
```

Monitor

```
// AXIS Monitor
initial begin
    repeat(NUM_EXP) begin

        sink.axis_pull_packet(out_data);
        {k_out, x_out} = queue.pop_back();

        exp_data = '0;
        foreach (k_out[r]) begin
            k_row_out = k_out[r];
            foreach (k_row_out[c])
                exp_data[r] = $signed(exp_data[r]) + $signed(k_row_out[c]) * $signed(x_out[c]);
            end

        assert (exp_data == out_data)
            $display("Outputs match: %d", exp_data);
        else $fatal(0, "Expected: %d != Output: %d", exp_data, out_data);

        end
    end
    $finish();
end
```

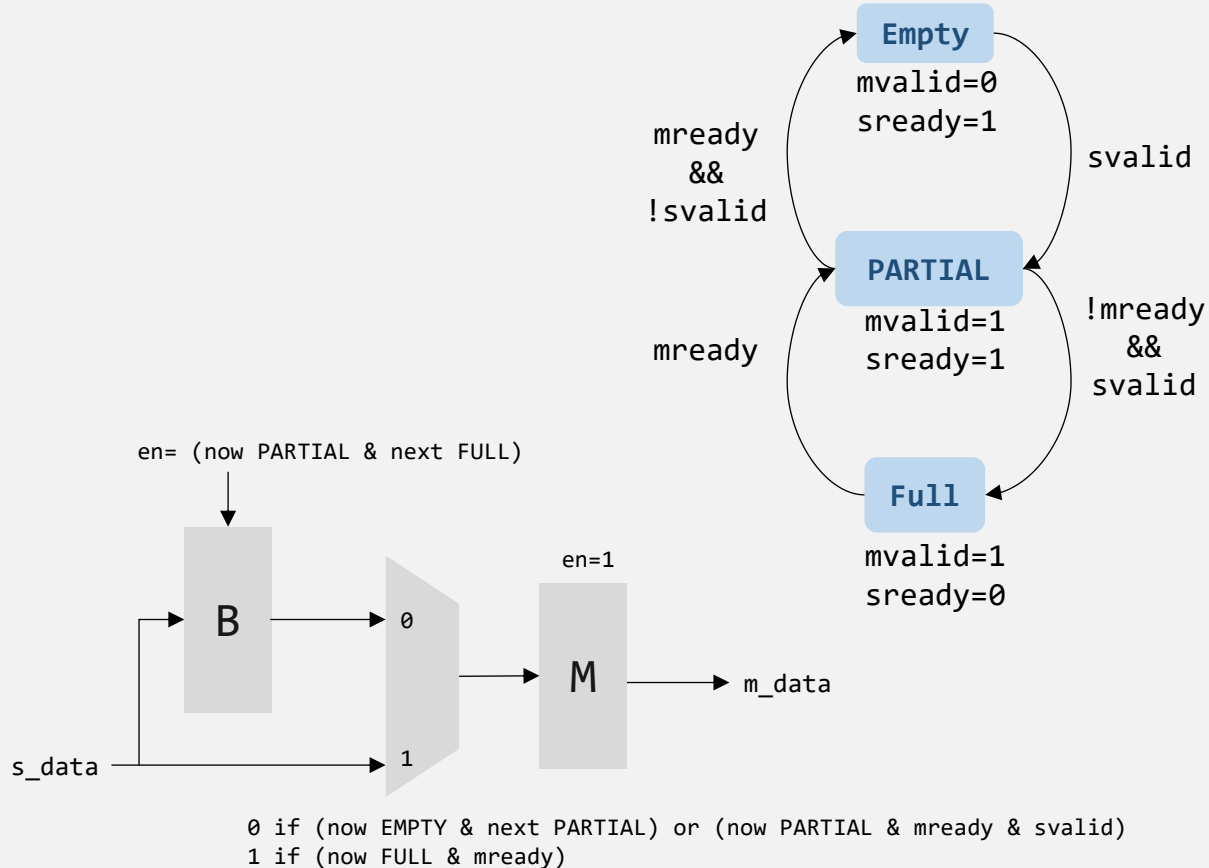
Software matmul

Compare

Skid Buffer

Abarajithan, 2025
abarajithan07@gmail.com

- **Problem:** data flow cannot stop in the same clock cycle without combinational logic
- **Solution:** “skid” the data into a buffer



```
module skid_buffer #(parameter WIDTH = 8)(
    input  logic clk, rstn, s_valid, m_ready,
    input  logic [WIDTH-1:0] s_data,
    output logic [WIDTH-1:0] m_data,
    output logic m_valid, s_ready
);
    enum {EMPTY, PARTIAL, FULL} state, state_next;
    always_comb begin
        state_next = state;
        unique case (state)
            EMPTY : if (s_valid) state_next = PARTIAL;
            PARTIAL: if (!m_ready && s_valid) state_next = FULL;
                   else if (m_ready && !s_valid) state_next = EMPTY;
            FULL  : if (m_ready) state_next = PARTIAL;
        endcase
    end
    always @(posedge clk)
        if (!rstn) state <= EMPTY;
        else state <= state_next;
end
```

State Machine: 3 procedure Why?
Having state_next as a wire gives a lot of flexibility

```
logic [WIDTH-1:0] buffer;
always @(posedge clk)
    if (!rstn) {m_valid, s_ready, buffer, m_data} <= 0;
    else begin
        m_valid <= state_next != EMPTY;
        s_ready <= state_next != FULL;
    end
```

s_ready registered, breaking the path

```
if (state == PARTIAL && state_next == FULL)
    buffer <= s_data;
```

data, valid also registered

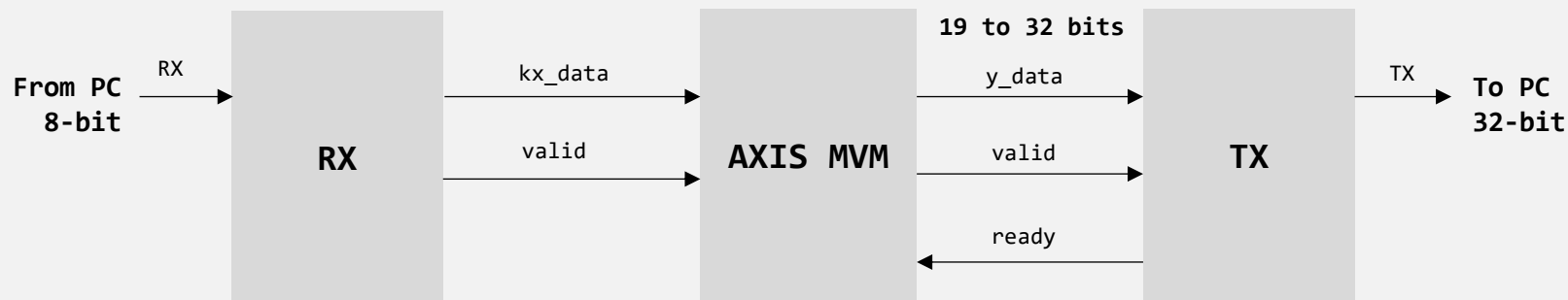
```
unique case (state)
    EMPTY : if (state_next == PARTIAL) m_data <= s_data;
    PARTIAL: if (m_ready && s_valid) m_data <= s_data;
    FULL  : if (m_ready) m_data <= buffer;
endcase
```

```
end
endmodule
```


Full System: UART + AXIS MVM

Abarajithan, 2025
abarajithan07@gmail.com

- Full system will be implemented on FPGA / ASIC
- Bit padding:
 - Data from PC: 8-bit
 - 8x8 matmul: $W_Y = 8+8+3 = 19$ bits
 - Sign extend it to 32 bits & sent to PC
- Ready:
 - RX does not accept m_ready (backpressure)
 - Fine, because input takes more time (more bits) than output
 - Always ready



Full System: UART + AXIS MVM

Abarajithan, 2025
abarajithan07@gmail.com

```
module mvm_uart_system #(
    parameter CLOCKS_PER_PULSE = 200_000_000/9600,
               BITS_PER_WORD    = 8,
               PACKET_SIZE_TX   = BITS_PER_WORD + 5,
               W_Y_OUT          = 32,
               R=8, C=8, W_X=4, W_K=3
)()
    input  clk, rstn, rx,
    output tx

    localparam W_BUS_KX = R*C*W_K + C*W_X,
               W_BUS_Y  = R*W_Y_OUT,
               W_Y       = W_X + W_K + $clog2(C);

    wire s_valid, m_valid, s_ready, m_ready;
    wire [W_BUS_KX-1:0] s_data_kx;

    uart_rx #(
        .CLOCKS_PER_PULSE (CLOCKS_PER_PULSE),
        .BITS_PER_WORD    (BITS_PER_WORD),
        .W_OUT (W_BUS_KX)
    ) UART_RX (
        .clk      (clk),
        .rstn     (rstn),
        .rx       (rx),
        .m_valid  (s_valid),
        .m_data   (s_data_kx)
    );
```

```
wire [R*W_Y -1:0] m_data_y;
axis_matvec_mul #(
    .R(R), .C(C), .W_X(W_X), .W_K(W_K)
) AXIS_MVM (
    .clk      (clk), .rstn (rstn),
    .s_axis_kx_tready(1'b1), // assume always ready
    .s_axis_kx_tvalid(s_valid),
    .s_axis_kx_tdata (s_data_kx),
    .m_axis_y_tready (m_ready),
    .m_axis_y_tvalid (m_valid),
    .m_axis_y_tdata  (m_data_y)
); // No packed arrays – need to manually flatten

// Padding to 32 bits to be read in computer
wire [W_Y -1:0] y_up [R-1:0];
wire [W_Y_OUT -1:0] o_up [R-1:0];
wire [R*W_Y_OUT-1:0] o_flat;
genvar r;
for (r=0; r<R; r=r+1) begin // Gets the r-th element of size W_Y
    assign y_up [r] = m_data_y[W_Y*(r+1)-1 : W_Y*r];
    assign o_up [r] = $signed(y_up[r]); // sign extend to 32b
    assign o_flat[W_Y_OUT*(r+1)-1 : W_Y_OUT*r] = o_up[r];
    // assign o_flat[W_Y_OUT*(r+1)-1 : W_Y_OUT*r] =
    // $signed(m_data_y[W_Y*(r+1)-1 : W_Y*r]); // One line
end
uart_tx #(
    .CLOCKS_PER_PULSE (CLOCKS_PER_PULSE),
    .BITS_PER_WORD    (BITS_PER_WORD),
    .PACKET_SIZE      (PACKET_SIZE_TX),
    .W_OUT             (W_BUS_Y)
) UART_TX (
    .clk      (clk), .rstn (rstn),
    .s_ready  (m_ready),
    .s_valid  (m_valid),
    .s_data   (o_flat),
    .tx       (tx)
);
endmodule
```

Full System Testbench: UART + AXIS MVM

Abarajithan, 2025
abarajithan07@gmail.com

```
module mvm_uart_system_tb;
  timeunit 1ns/1ps;
  localparam R=8, C=8, W_X=8, W_K=8,
    CLOCKS_PER_PULSE = 4, //200_000_000/9600,
    BITS_PER_WORD = 8,
    W_Y = W_X + W_K + $clog2(C),
    W_BUS_KX = R*C*W_K + C*W_X,
    W_BUS_Y = R*W_Y,
    N_WORDS_KX = W_BUS_KX/BITS_PER_WORD,
    N_WORDS_Y = W_BUS_Y /BITS_PER_WORD,
    PACKET_SIZE_TX = BITS_PER_WORD+5,
    CLK_PERIOD = 10,
    NUM_EXP = 10;

  logic clk=0, rstn=0, rx=1, tx;
  initial forever #(CLK_PERIOD/2) clk <= !clk;

  mvm_uart_system #(
    .CLOCKS_PER_PULSE (CLOCKS_PER_PULSE), //200_000_000/9600
    .BITS_PER_WORD (BITS_PER_WORD),
    .PACKET_SIZE_TX(PACKET_SIZE_TX),
    .R(R),.C(C),.W_X(W_X),.W_K(W_K)) dut (.*);
```

```
int rx_bits;
initial forever begin
  rx_bits = 0;
  wait(!rx);
  for (int n=0; n<BITS_PER_WORD+2; n++) begin
    rx_bits += 1;
    repeat (CLOCKS_PER_PULSE) @(posedge clk);
  end end
```

Count UART bits
To easily identify
in waveform

// Driver

Send data into RX

```
logic [N_WORDS_KX-1:0][BITS_PER_WORD-1:0] s_data;
logic [BITS_PER_WORD+2-1:0] s_packet;
logic [N_WORDS_KX-1:0][BITS_PER_WORD-1:0] queue_kx [$];
```

initial begin

```
assert (W_BUS_KX % BITS_PER_WORD == 0);
assert (W_BUS_Y % BITS_PER_WORD == 0);
$dumpfile("dump.vcd"); $dumpvars;
```

Check if widths
are divisible

```
repeat(2) @(posedge clk) #1;
rstn = 1;
```

repeat (NUM_EXP) begin

Create and push random data

```
for (int iw=0; iw<N_WORDS_KX; iw++) begin
  s_data[iw] = $urandom_range(2**BITS_PER_WORD-1);
  s_packet = {1'b1, s_data[iw], 1'b0};
```

```
  repeat ($urandom_range(1,20)) @(posedge clk);
```

```
  for (int ib=0; ib<BITS_PER_WORD+2; ib++)
    repeat(CLOCKS_PER_PULSE) begin
      #1 rx <= s_packet[ib];
      @(posedge clk);
    end
```

```
end
queue_kx.push_front(s_data);
repeat ($urandom_range(1,100)) @(posedge clk);
```

```
end
end
```

Full System Testbench...

Abarajithan, 2025
abarajithan07@gmail.com

```
int tx_bits;                                Count UART bits
initial forever begin                       To easily identify
    tx_bits = 0;                            in waveform
    wait(!tx);
    for (int n=0; n<PACKET_SIZE_TX; n++) begin
        tx_bits += 1;
        repeat (CLOCKS_PER_PULSE) @(posedge clk);
    end
end
```

// Monitor

```
logic [C-1:0][W_X-1:0] x_out;
logic [R-1:0][C-1:0][W_K-1:0] k_out;
logic [C-1:0][W_K-1:0] k_row_out;
logic [R-1:0][W_Y-1:0] exp_data;
logic [N_WORDS_Y -1:0][BITS_PER_WORD-1:0] m_data;
logic [BITS_PER_WORD-1 :0] m_packet;
```

```
initial begin
    repeat (NUM_EXP) begin
        m_data <= 'x;
        for (int iw=0; iw<N_WORDS_Y; iw++) begin // get each word

            wait(!tx);
            repeat (CLOCKS_PER_PULSE/2) @(posedge clk); // go to middle of start bit

            for (int ib=0; ib<BITS_PER_WORD; ib++) begin
                repeat (CLOCKS_PER_PULSE) @(posedge clk); // go to middle of data bit
                m_packet[ib] = tx;
            end
            m_data[iw] = m_packet;

            for (int ib=0; ib<PACKET_SIZE_TX-BITS_PER_WORD-1; ib=ib+1) begin
                repeat (CLOCKS_PER_PULSE) @(posedge clk);
                assert (tx == 1) else $error("Incorrect end bits/padding");
            end
        end
    end
end
```

```
{k_out, x_out} = queue_kx.pop_back();
```

```
// Matrix Vector Multiplication in software
exp_data = '0;
foreach (k_out[r]) begin
    k_row_out = k_out[r];
    foreach (k_row_out[c])
        exp_data[r] = $signed(exp_data[r]) + $signed(k_row_out[c]) * $signed(x_out[c]);
end
```

```
// Compare
assert (exp_data == m_data)
    $display("Outputs match: %d", exp_data);
else $fatal(0, "Expected: %d != Output: %d", exp_data, m_data);
```

```
end
$finish();
end
```

Compare

Get each output vector

Software matmul

Data Types – Summary

- Literals : `result = 16'd119 + 24'h2F + 'd4 - 3'd5 + 8'b00_00_0110;`
- Fill Literal : `'0, '1, 'x, 'z`
- Type : bit (2-state), logic (4-state), Verilog wire & reg are obsolete
- Variable (var) : single driver
- Net types : `wire = tri (multiple drivers), supply0, supply1`
- Signed / unsigned : for automatic sign extension
- Synthesizable datatypes

```
logic      : var logic unsigned
integer    : var logic unsigned [31:0]
bit        : var bit
int        : var bit    unsigned [31:0]
byte       : var bit    unsigned [ 7:0]
```
- Generate loops : `genvar i; for(i= 0; i <10; i++)`
- Non-synthesizable datatypes : `real (double), shortreal (float), string`
- Vector (packed array) : `logic [7:0] vec_1;`
- Unpacked Array : `logic [7:0] arr_1 [15:0][16];`
- Slicing & selection : `assign a = vec_1[3]; assign b = vec_1[5:2]`
- Typedef

```
typedef logic signed [15:0] short_t;    short_t a,b,c
typedef logic [7:0] mat_4x4_t [4][4];   mat_4x4_t a, b, c;
```
- Queues (non-synth) : `int q1 [$]; q1.push_front(6); q1.push_back(7); q.pop_front();`
- Display, write (non-synth) : `$display("anything -- %p", any); $write("hello %p", any);`
- Structs, unions (synth) : `struct {state_reg, state_next, count_reg} fsm_s;`

SystemVerilog Gotchas

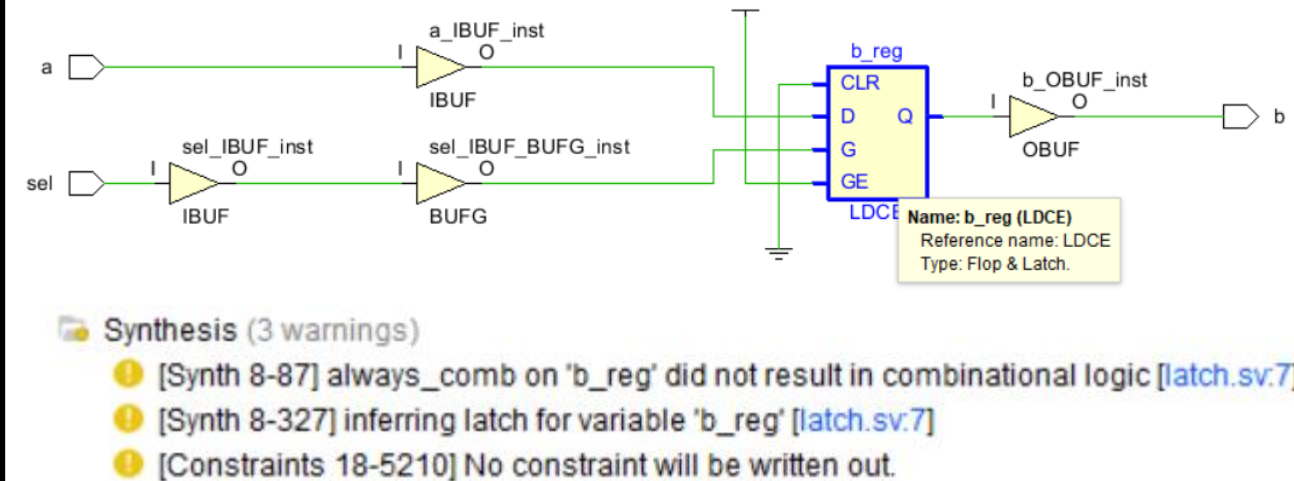
Abarajithan, 2025
abarajithan07@gmail.com

- Signed arithmetic – all operands have to be signed or **\$signed()**. Even 0 is unsigned
- Continuous assignment: **wire x = y;** vs Testbench/FPGA-RTL initialization: **logic x = y;**
- Testbench: never read & write at clock edge. Always: **@(posedge clk) #1 x <= y; \$display("%d", z);**
- Typos in instantiations will silently “infer 1-bit nets” : **logic [7:0] x,y,z; mul M (.x(x), .y(u), .z(z));**
- If unspecified, default base of literals is 10: if (x==10)
- Default sign of unbased vs based literals: **logic signed [7:0] x=-5; x+1=>-4; x+1'b1 => 252**
- Filling all ones: **x = 1** vs **x = '1**
- Concat vs array assignment: **wire [1:0][31:0] x; x = {1'b1, 1'b0}; => {0,2} x = '{1'b1, 1'b0}; => {1, 0}**
- Port connection width mismatch
- Accidentally driving an input port
- Incomplete sensitivity list: **always (s, a) y = s ? a : b;** Use **always_comb**, or **always (*)**

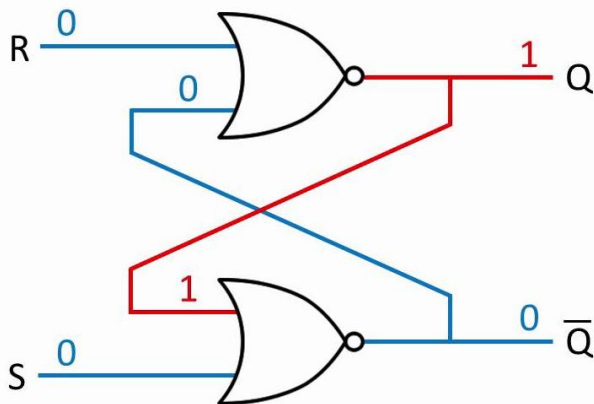
Latch Inference

- Combinational storage
- Level sensitive, not edge
- Unintentional inference causes
 - race conditions,
 - meta-stability and
 - inefficient resource usage

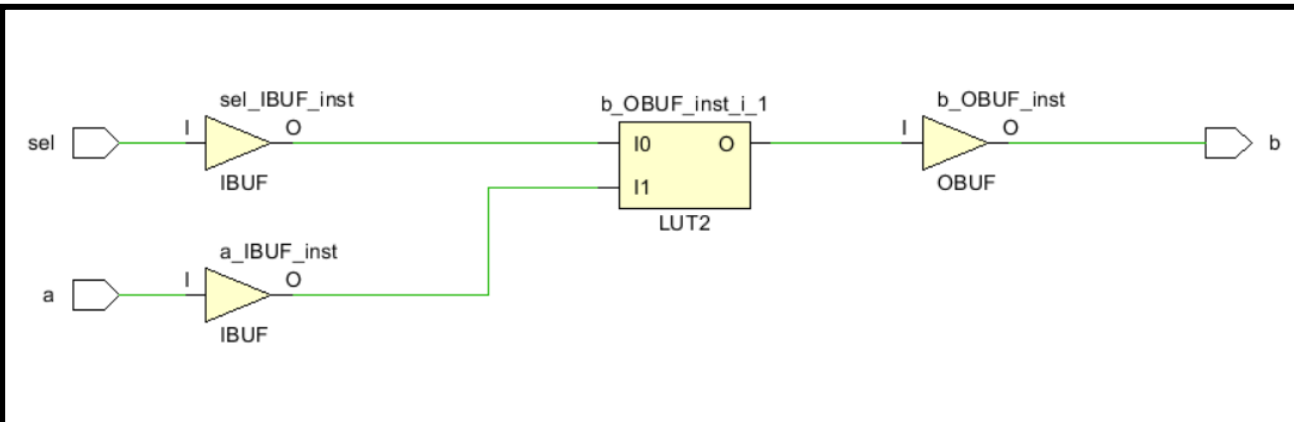
```
module latch(  
    input logic sel,  
    input logic a ,  
    output logic b  
);  
    always_comb  
        if (sel) b = a;  
        // else b = 0;  
endmodule
```



S	R	Q	$\bar{Q}$
0	0	1	0
0	0	0	1
0	1	0	1
1	0	1	0
1	1	0	0



MUX INFERRED AS LUT IN SYNTHESIS



RTL: Best practices

Abarajithan, 2025
abarajithan07@gmail.com

- Avoid modifying the clock signal in RTL. Use clock enables instead.
- Avoid combinational feedback
- Use `always_ff`, `always_comb`, `always_latch` over `always`
- Blocking (=) for combinational, Non-blocking (<=) for flip flops
- If using `always`, include all changing signals in sensitivity list
- Avoid latch inference in comb: always have a `default` for `case()` and `else` for an `if`
- Beware of vector size mismatch!

	<pre>clk = ~clk ; clk_reg = sel & clk;</pre>
	<pre>always_ff @(posedge clk) if (sel) data_reg <= A;</pre>
	<pre>always_comb if (sel) data_out = A;</pre>
	<pre>always_comb if (sel) data = A; else data = B;</pre>
	<pre>assign a = a + b; always_comb a = a + b;</pre>

FPGA	ASIC
• Initialize registers at power-up as: <pre>logic [7:0] word = 8'b0; logic [7:0] data [3][3] = '{default:0};</pre> • Global resets are expensive • If necessary, use synchronous resets	• Power-up initialization is not possible. • Use asynchronous, global, active-low resets <pre>always_ff @(posedge clk or negedge rstn) begin if (!rstn) data <= 0; else // do something end</pre>

Guidelines for Clean Code

1. Input to output in design => Left to right in diagram => top to bottom in code

2. Naming:

- logic/wires/reg, functions : `var_name` (wire, reg..), `func_name`
- Classes : `Class_Name`
- Active low signals : `rstn` / `rst_n`
- Typedefs : `type_name_t`
- Enums : `enum_name_e`
- Constants : `PARAM`, `ENUM_VAL`, ``MACRO_NAME`

Guidelines for Clean Code

Abarajithan, 2025
abarajithan07@gmail.com

- Do not use `/**/` for comments in code body. Instead, use `//`
- Used named or `.*` connections. Do not use port order connections
- Use logic type everywhere in RTL design. Only use wire for one-line assignments.
- Do not use reg
- Use ANSI-C style port declarations (ones we use in examples). Declare both input & output as logic
- Use enums for states
- Use logical operators (`&&`, `||`, `!`) even for 1 bit signals, rather than bitwise (`&`, `|`, `~`)
- Avoid mixing signed & unsigned variables in expressions. Make sure either all are unsigned, or all are signed.
- Declare functions used in RTL as automatic

Guidelines for Clean Code

Abarajithan, 2025
abarajithan07@gmail.com

- Ensure both left & right hand side have same width in assignments (see warnings)
- Use `always_comb`, `always_ff`, `always_latch`. Never use `always@`
- Only use `=` for combinational logic, `<=` for sequential logic. Never mix in RTL design
- Use `unique case` instead of `if-else` for parallel multiplexers
- Be consistent with active high/low reset throughout the design

TCL Scripting

Abarajithan, 2025
abarajithan07@gmail.com

```
set PROJ_NAME sv_examples
set PROJ_FOLDER sv_examples
set SOURCE_FOLDER ../src
```

Assign to a variables

Tutorial:

tcl.tk/man/tcl8.5/tutorial/tcltutorial.html

Create project

Refer variables

```
create_project $PROJ_NAME ./ $PROJ_FOLDER -part xc7z020c1g484-1
set_property board_part xilinx.com:zc702:part0:1.4 [current_project]
```

Add sources

```
add_files $SOURCE_FOLDER
```

add_sub IP

```
set IP_NAME c_addsub_0
create_ip -name c_addsub -vendor xilinx.com -library ip -version 12.0 -module_name $IP_NAME
set_property -
```

```
dict [list CONFIG.A_Width {8} CONFIG.B_Width {8} CONFIG.Add_Mode {Add_Subtract} CONFIG.Latency_Configuration {Manual} CON
FIG.CE {false} CONFIG.Out_Width {8} CONFIG.Latency {1} CONFIG.B_Value {00000000}] [get_ips $IP_NAME]
```

Refer variables

```
generate_target {instantiation_template} [get_files $PROJ_FOLDER/$PROJ_NAME.srcs/sources_1/ip/$IP_NAME/$IP_NAME.xci]
set_property generate_synth_checkpoint 0 [get_files $PROJ_FOLDER/$PROJ_NAME.srcs/sources_1/ip/$IP_NAME/$IP_NAME.xci]
export_ip_user_files -of_objects [get_files $PROJ_FOLDER/$PROJ_NAME.srcs/sources_1/ip/$IP_NAME/$IP_NAME.xci] -no_script -sync -force -quiet
export_simulation -of_objects [get_files $PROJ_FOLDER/$PROJ_NAME.srcs/sources_1/ip/$IP_NAME/$IP_NAME.xci] -directory $PROJ_FOLDER/$PROJ_NAME.ip_user_files/sim_scripts -
ip_user_files_dir $PROJ_FOLDER/$PROJ_NAME.ip_user_files -ipstatic_source_dir $PROJ_FOLDER/$PROJ_NAME.ip_user_files/ipstatic -
lib_map_path [list {modelsim=$PROJ_FOLDER/$PROJ_NAME.cache/compile_simlib/modelsim} {questa=$PROJ_FOLDER/$PROJ_NAME.cache/compile_simlib/questa} {riviera=$PROJ_FOLDER/$PR
OJ_NAME.cache/compile_simlib/riviera} {activehdl=$PROJ_FOLDER/$PROJ_NAME.cache/compile_simlib/activehdl}] -use_ip_compiled_libs -force -quiet
```